

Verifying Agent Repairs: Evidence Quality, Deployment Gains, and Cost

Anonymous ACL submission

Abstract

Self-improving agent loops change an agent’s harness (its prompts, tool descriptions and control code) and keep a change only when verification evidence says it helps: a replay of the failure, an LLM judge’s verdict, reruns of solved tasks, or a full re-evaluation. We ask which evidence predicts a change’s gain when it is deployed on independent tasks, and whether the evidence is worth its cost, counted in agent episodes. In one round of attribution-driven repair of a DeepSeek-V4-Flash agent on τ^2 -bench, mostly of injected faults, no patch set chosen by up to ten verification policies beat applying the attributed patches unverified, in a test with little power. We therefore built on RRSI, a regularised self-improvement loop that edits harness code, collected seven kinds of evidence for each of the 47 candidate changes it measured over 20 rounds in each of two domains, and deployed every candidate on held-out tasks under a registered analysis. Most candidates lowered held-out success. Without a check for harness errors, an LLM judge and replays without null replays did as badly as no verification, because they accepted a change that crashed the harness in half of its episodes. With the same error check for every rule, RRSI’s full re-evaluation had the highest decision value, +0.22 points of held-out success per round (90% interval $[-0.06, +0.62]$), but evidence at a third of its cost was neither shown non-inferior nor shown inferior to it; on RRSI’s trajectory our design resolves only differences of about 0.6 points per round. Our results thus identify failure modes of verification and the measurement resolution that comparing it requires, rather than a ranking of evidence by predictive quality.

1 Introduction

A growing family of systems improves LLM agents from their own failures. The agent runs, an analyst model reads the failed trajectories, a proposer

changes the agent’s *harness*, the prompts, tool descriptions and code around a frozen model, and the loop keeps the change or discards it (Shinn et al., 2023; Zhao et al., 2024; Hu et al., 2025; Zhang et al., 2025a; Agrawal et al., 2026; Chen et al., 2026; Zhang et al., 2026b; Google Cloud AI Research and collaborators, 2026). The check that decides is where these systems differ most: some replay the failure with the change applied (Ma et al., 2025; Lin et al., 2026; Shah, 2026), some ask an LLM judge, some re-run tasks that used to pass (Chen et al., 2026), and RRSI re-evaluates every candidate twice on its whole training set (Google Cloud AI Research and collaborators, 2026). What every check is meant to estimate is a change’s *deployment gain*, how much it raises success on tasks the loop has not seen, and that gain can differ from what the loop observes on its own tasks: RRSI reports in-distribution gains that shrink out of distribution (Google Cloud AI Research and collaborators, 2026).

This paper asks two questions about verification evidence. *Which evidence predicts a change’s deployment gain?* And *is the evidence worth what it costs?* Answering the first needs ground truth that verification does not provide: deploying the changes, from the first step, on independent tasks. Answering the second needs an account of what verification costs and what the change it selects is worth over a deployment horizon, which we write as $\text{Net}(N)$ (§3). We price every piece of evidence in agent episodes and answer both questions in two studies.

Study 1 (§4, completed) is one round of attribution-driven repair of a DeepSeek-V4-Flash agent on the retail and airline domains of τ^2 -bench (Barres et al., 2025), with patches that rewrite policy sections, tool descriptions and the step limit. We record intervention banks of targeted replays, null replays, judge checks and regression runs for 108 patches per domain, let eleven verification poli-

cies choose patch sets from them, and deploy the sets on held-out tasks. Four findings stand out. Applying the attributed patches without verification raised held-out success by 11.7 (retail) and 6.7 (airline) points, mostly on episodes with injected faults, and no verified set did better. Replay evidence ranked the deployed sets poorly, but the differences it predicted between sets were below the noise of our deployment measurements. No evidence was cheap: judge checks cost 22–72% of a replay and were at best about as informative per token, and a fifth of unpatched replays succeed anyway. Finally, verification cost at most ten episodes, whereas over a thousand deployed episodes, with a success worth ten episodes of compute, the 90% interval of its net worth spans 600 to 1,800 episodes, so its worth turns on its quality, not its price.

Study 2 (§5, registered) tests evidence quality with the power and the modification space Study 1 lacked. We build on RRSI’s open-source loop (Google Cloud AI Research and collaborators, 2026), whose proposer edits harness code over many rounds of natural failures, port it to DeepSeek models and τ^2 -bench, make targeted replays work for code-level changes, and replace RRSI’s selection step with each kind of evidence at equal budget. Every candidate and incumbent is deployed on held-out tasks, and the study’s endpoints, decision rule, analysis code and power analysis were fixed before any held-out episode was run. Over 40 rounds, most of the 47 measured candidates lowered held-out success. Without a check for harness errors, an LLM judge and replays without null replays did as badly as no verification, because they accepted a change whose bug crashed the harness in half of its episodes. With the same check for every rule, RRSI’s full re-evaluation had the highest decision value, but cheaper evidence was neither shown non-inferior nor inferior to it, and the design resolves only per-round differences of about 0.6 points or more.

Our contributions are a deployment-grounded evaluation of verification evidence with an explicit cost account; the measurements of Study 1; and the RRSI-based harness, replay mechanism, registered design and results of Study 2, which we release together with the environments, banks and evidence. The multi-fidelity allocation method we started from, which assumed that judge checks are cheap, is reported as a design study in Appendix O.

2 Related Work

Self-improving agents and how they verify. Reflexion, Self-Refine and ExpeL turn feedback into verbal lessons (Shinn et al., 2023; Madaan et al., 2023; Zhao et al., 2024); ProTeGi, OPRO, TextGrad and DSPy rewrite prompts (Pryzant et al., 2023; Yang et al., 2024; Yuksekogonul et al., 2025; Khattab et al., 2024); ADAS, the Darwin Gödel Machine and GEPA search over designs (Hu et al., 2025; Zhang et al., 2025a; Agrawal et al., 2026); ACE evolves a context playbook through incremental updates (Zhang et al., 2026b). Harness-Fix repairs harness code and promotes a bundle of repairs only if it solves more validation tasks while breaking at most two (Chen et al., 2026); Bayesian-Agent applies threshold rules to posteriors over skills (Wu et al., 2026). RRSI regularises harness self-improvement: it bounds and anneals the number of edits per candidate, screens benchmark-specific edits with a critic, prunes stale edits, and selects candidates by full re-evaluation against a calibrated noise floor (Google Cloud AI Research and collaborators, 2026). These systems report the success of the final agent; we evaluate the evidence behind each decision, and Study 2 uses RRSI’s code as its loop.

Failure attribution. Who&When and its extension show that locating the responsible agent and step is far from solved (Zhang et al., 2025b; Liu et al., 2026). AgenTracer trains an attribution model on counterfactual replays and injected faults (Zhang et al., 2026a); AgentDebug and ErrorProbe pair taxonomies or dependency tracing with corrective feedback (Zhu et al., 2025; Li et al., 2026); MAST and AdaMAST give attribution a vocabulary (Cemri et al., 2025, 2026).

Verification by intervention. DoVer validates hypotheses by editing a message or plan and re-running (Ma et al., 2025); REFLECT tests patches by targeted replay (Lin et al., 2026); CausalFlow and Causal Agent Replay score step interventions causally (Bonagiri et al., 2026; Shah, 2026); Agent-DebugX gates reruns by policy (Zhu et al., 2026). None of them measures whether its evidence predicts deployment or what the evidence costs. Program repair has long warned that a patch passing the failing test may not fix the fault (Smith et al., 2015).

3 Verification Evidence and Deployment Gain

Changes and their deployment gain. A loop holds an incumbent harness h and proposes a change c : a rewritten prompt section or tool description, a code edit, or a set of these. Its *deployment gain* $\Delta p(c)$ is the difference in success rate between h with c and h alone, measured from the first step on tasks independent of everything the loop saw. A verification rule spends C_{verify} episodes on evidence, forms an estimate $\hat{\Delta}$ of the change’s effect and accepts the change if the estimate clears a threshold.

Evidence. We study the evidence that self-improvement loops use. *None* accepts the proposed change. An *LLM judge* reads the change and the failures it targets and predicts its effect, at a cost in tokens but no episodes; a *single-step check* regenerates the failing step with the change and asks a judge whether the error is gone. A *targeted replay* resumes a failed trajectory from the failing step with the change and succeeds with probability $b + (1 - b)q$, where q is the share of such failures the change repairs; a *null replay* does the same without the change and measures the *spurious recovery* rate b , without which replay success cannot be interpreted. A *regression run* is a previously solved task, from the first step, with the change; a *sample* or *full evaluation* runs candidate and incumbent on a random sample of, or all, training tasks. Replays and checks see only the failures a change targets; only regression runs and evaluations see harm to tasks that already succeed.

Is the evidence worth its cost? Compare a rule E with a reference rule, here applying changes without verification. Suppose E ’s choices change the deployed agent’s success by Δp and its running cost by Δc_{run} per episode, that E spends C_{verify} episodes, and that it changes the one-off cost of reviewing and maintaining the applied changes by ΔC_{change} . Over the next N deployed episodes E is worth

$$\text{Net}(N) = N(v\Delta p - \Delta c_{\text{run}}) - C_{\text{verify}} - \Delta C_{\text{change}}, \quad (1)$$

with every cost in episodes of the unmodified agent’s running cost and v the value of one success in the same unit. The one-off costs do not grow with N , so once Nv is large compared with C_{verify} the sign of $\text{Net}(N)$ is the sign of $v\Delta p - \Delta c_{\text{run}}$:

evidence pays if the changes it selects are better, or cheaper to run. Its worth then rests on its *quality*, which we measure in two ways: how well its estimates correlate with, and are calibrated against, measured deployment gains, and the *decision value* of a rule that uses it, the mean deployment gain of the changes it accepts.

An accounting decomposition. Appendix A splits a patch’s net effect into repair, spurious recovery and regressions, and bounds the value of perfect verification; it shows where verification could add value, not a decision rule.

4 Study 1: One Round of Attribution-Driven Repair

Agent and environments. The agent is DeepSeek-V4-Flash, with the provider’s default sampling, on the retail and airline domains of τ^2 -bench (Yao et al., 2024; Barres et al., 2025), which have an LLM user simulator and database-state rewards. Attribution, taxonomy induction (Wan et al., 2024), patch generation and judging use DeepSeek-V4-Pro. The patchable components are the sections of the domain policy, every tool description and the step limit: 24 components on retail and 21 on airline. Each training episode runs clean or with one injected fault written as a plausible authoring mistake (retail: a policy section allowing gift-card refunds, a cancellation rule without the reason check, a product-tool description that confuses item and product ids, a tight step limit; airline: a wrong bag allowance, a 7-day instead of 24-hour cancellation window, a missing basic-economy restriction, proactive compensation, a tight step limit). Clean episodes contribute natural failures, 22 of 137 on retail and 28 of 112 on airline. A third environment, ShopDesk, is described in Appendix L.

Patches and banks. The analyst groups failures into categories and blames components. For 36 cells (category, component) per domain, the blamed cells in order of blame plus random unblamed ones, it writes three patches each. For every patch we record 20 targeted replays from the attributed failing step, each paired with a single-step check; for every category, 20 null replays; and for every patch, 4 regression runs on solved training tasks, extended to 20 for the 121 patches that any policy accepted. An unpatched baseline of 10 runs on each of the 111 regression tasks gives task-

matched estimates of harm. Policies run offline on these *intervention banks*, with every experiment priced in measured tokens relative to a full replay.

Policies and deployment. Eleven policies spend a budget of episodes from the banks and output a patch set (Appendix C). *Apply attributed* uses no evidence: for each category it applies the first patch of the most-blamed component and of any component with at least 40% of the blame. *Replay-each* accepts a hypothesis on two successes in three replays, as do-then-verify methods do (Ma et al., 2025); two HarnessFix-style gates require regression runs to break nothing, per patch or on the whole bundle (Chen et al., 2026); uncertainty sampling runs with and without cheap checks; four variants of CARVE, the multi-fidelity design we started from, and a net-effect allocation that charges regressions complete the list. We apply each policy’s budget-10 set, from five seeds and reduced to one patch per component, to unseen tasks: 40 retail tasks $\times$ 3 episodes (38 clean, 82 with an injected fault) and 20 airline tasks $\times$ 3 (25 clean), for five policies on airline. Retail was the development domain for every method.

4.1 Deployment gains

Table 1 reports absolute held-out success. Applying the attributed patches without verification raised success from 75.8% to 87.5% on retail (+11.7 points, 90% interval [4.5, 18.8]) and from 68.3% to 75.0% on airline (+6.7, [−3.3, 16.6]). No verified set did better: their differences from the unverified set range from −4.3 to +0.3 points on retail and from −4.0 to −0.3 on airline. This is not an equivalence result. The intervals reach +6.0 points on retail and +8.5 on airline; with a margin of 5 points, chosen after the fact, applying the attributed patches is non-inferior to 9 of the 10 verified sets on retail (all but CARVE with replays only) and to the two HarnessFix-style sets on airline. Most of retail’s gain comes from fault-injected episodes: on its 38 clean episodes the unverified set gains +5.3 points [−5.1, 15.6], and on 158 clean episodes, 120 of them new, +4 [−3, 10] (Appendix J).

4.2 Does the evidence predict deployment?

Replays measure repair from the failing step, whereas deployed patches act from the first step and on every task. For every distinct deployed set we predict its gain over the unpatched agent from its patches’ replays as $\hat{F} \sum_i f_i [1 - \prod_k (1 - \hat{q}_k)]$, with

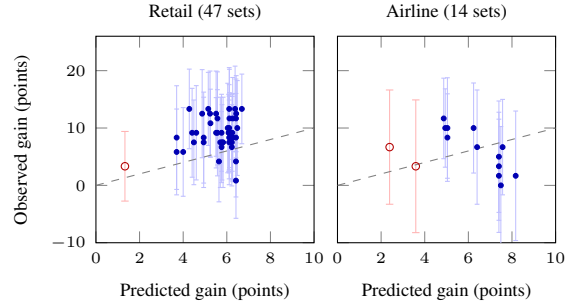


Figure 1: Each distinct deployed patch set’s gain over the unpatched agent as predicted from its bank replays (net of spurious recovery) against its observed held-out gain, with 90% intervals. Filled: sets containing the step-limit patch; open: sets without it. Dashed: $y = x$. Correlations are 0.23 (retail) and −0.49 (airline).

$\hat{F}$ the held-out failure rate, f_i the category shares and $\hat{q}_k = (p_k - b_i)/(1 - b_i)$ the replay success p_k net of the category’s spurious recovery b_i . Figure 1 compares the predictions with the observed gains. They correlate weakly, at $r = 0.23$ over 47 distinct retail sets and −0.49 over 14 airline sets; predictions from the judge checks, calibrated to replay success, do no better ($r = 0.14$ and −0.05). In level, replays predict airline’s gains (6.0 points on average against 6.1 observed) but not retail’s (5.6 against 9.1).

The ranking test has little power, for a reason that matters beyond this study. Replay predictions differ between sets by a standard deviation of 1.0 point on retail and 1.7 on airline, whereas a set’s held-out success measured on 120 (60) episodes has a run-to-run standard error of 2.6 (3.8) points; the observed gains spread by 2.8 (3.7), no more than that noise. A perfect predictor of differences this small would be indistinguishable from a useless one. No single patch’s deployed effect is resolved either (Appendix J). Testing whether evidence predicts deployment needs candidates that differ by more than the deployment noise and more precise deployment measurements; Study 2 is designed for both.

4.3 What the evidence costs

Table 2 collects the measurements. *Judge checks are not cheap*. A single-step check costs 22–72% of a full replay in tokens, and its information per token is at best similar to a replay’s: against a patch’s replay success rate its efficiency ratio is 0.88 [0.46, 1.53] on retail and 1.25 [0.19, 3.95] on airline, and against single replays it is below 1 for

Patch set (budget 10)	Retail (40 tasks $\times$ 3)					Airline (20 tasks $\times$ 3)				
	All	Clean	–Apply [90%]	#p	C	All	Clean	–Apply [90%]	#p	C
Unpatched	75.8	81.6		0.0		68.3	76.0		0.0	
Apply attributed	87.5	86.8	–	7.0	0	75.0	76.0	–	5.0	0
Replay-each	83.5	86.8	–4.0 [–9.8, 1.8]	6.0	10					
HarnessFix-style, per patch	83.2	90.0	–4.3 [–12.8, 4.2]	0.8	9	71.0	76.8	–4.0 [–11.9, 3.9]	1.0	8
HarnessFix-style, bundle	84.8	86.8	–2.7 [–5.0, –0.4]	5.6	6	71.7	76.8	–3.3 [–9.3, 2.6]	3.0	6
Uncertainty sampling	85.8	86.3	–1.7 [–6.9, 3.5]	5.8	10					
Uncertainty + checks	84.7	82.1	–2.8 [–8.4, 2.7]	5.8	10					
CARVE, replay only	87.8	86.8	0.3 [–5.3, 6.0]	6.0	10					
CARVE	83.8	80.5	–3.7 [–9.9, 2.6]	5.2	10	74.3	72.0	–0.7 [–8.8, 7.5]	4.2	10
CARVE, measured check	84.8	86.3	–2.7 [–7.6, 2.3]	6.0	10					
CARVE, robust check	86.3	83.7	–1.2 [–6.3, 4.0]	6.0	9					
Net-effect	83.2	83.2	–4.3 [–9.9, 1.3]	3.4	10	74.7	76.0	–0.3 [–9.1, 8.5]	3.6	10

Table 1: Held-out success (%) of the patch set each policy selects with a budget of 10 episodes, mean over five policy seeds. *All*: every held-out episode (retail: 82 of 120 with an injected fault; airline: 35 of 60). *Clean*: episodes without an injected fault (38 and 25). *–Apply*: paired difference from Apply attributed over all episodes, in points, with a task-clustered 90% interval. *#p*: patches deployed; *C*: verification episodes spent.

	ShopDesk	Retail	Airline
Analyst accuracy (audited)	0.72	0.41	0.29
Effective edges among blamed cells	–	0.18	0.33
Judge check: cost / replay	0.72	0.28	0.23
sens. / FPR, vs. single replay	0.57 / 0.21	0.37 / 0.11	0.41 / 0.13
sens. / FPR, vs. patch replay rate	–	0.49 / 0.05	0.55 / 0.07
efficiency ratio vs. replays	–	0.88 [0.46, 1.53]	1.25 [0.19, 3.95]
Spurious recovery b (range)	–	0.23 (0–0.50)	0.20 (0–0.50)
Repair q of effective edges (median)	–	0.50	0.41
Replays to decide an edge	–	5	7
Unpatched failure on solved tasks	–	0.15	0.12
Mean harm $\bar{\rho}$ (points)	–	–1.0 [–3.4, 1.5]	–0.6 [–3.3, 1.9]
Spread of ρ_k (points)	–	2.6 [0.2, 5.3]	2.1 [0.2, 4.5]
Patches harmful / helpful (95%)	–	2 / 5	3 / 4

Table 2: What the evidence costs and shows. The efficiency ratio is the judge’s break-even cost ratio divided by its actual cost (Proposition 4); values above 1 favour judge checks (Appendix H). Harm rows: excess failure rate of a patch on previously solved tasks under the task-level binomial model (Appendix I), posterior mean and 90% interval; the last row counts patches whose 20 regression runs differ significantly from the task-matched baseline.

Verified set	Δp [90%]	Δc_{run}	C	Net(1000) [90%], episodes	
				$v = 1$	$v = 10$
<i>Retail</i>					
Replay-each	–4.0 [–9.8, 1.8]	–0.06	10	12 [–217, 242]	–348 [–966, 271]
HarnessFix-style, per patch	–4.3 [–12.8, 4.2]	–0.03	9	–18 [–204, 167]	–408 [–1273, 456]
HarnessFix-style, bundle	–2.7 [–5.0, –0.4]	–0.00	6	–31 [–244, 183]	–271 [–583, 42]
Uncertainty sampling	–1.7 [–6.9, 3.5]	–0.07	10	41 [–189, 271]	–109 [–676, 458]
Uncertainty + checks	–2.8 [–8.4, 2.7]	0.02	10	–56 [–285, 173]	–311 [–907, 286]
CARVE, replay only	0.3 [–5.3, 6.0]	–0.02	10	10 [–220, 240]	40 [–568, 648]
CARVE	–3.7 [–9.9, 2.6]	–0.04	10	–8 [–227, 210]	–338 [–999, 322]
CARVE, measured check	–2.7 [–7.6, 2.3]	–0.01	10	–24 [–254, 206]	–264 [–810, 282]
CARVE, robust check	–1.2 [–6.3, 4.0]	0.02	9	–37 [–263, 188]	–142 [–704, 420]
Net-effect	–4.3 [–9.9, 1.3]	0.05	10	–100 [–307, 106]	–490 [–1083, 103]
<i>Airline</i>					
HarnessFix-style, per patch	–4.0 [–11.9, 3.9]	–0.04	8	–7 [–226, 212]	–367 [–1185, 451]
HarnessFix-style, bundle	–3.3 [–9.3, 2.6]	–0.06	6	23 [–209, 254]	–277 [–911, 356]
CARVE	–0.7 [–8.8, 7.5]	–0.11	10	96 [–156, 349]	36 [–814, 887]
Net-effect	–0.3 [–9.1, 8.5]	–0.08	10	68 [–194, 330]	38 [–875, 951]

Table 3: Net(N) of each verified set against Apply attributed (Eq. 1), at $N = 1000$ deployed episodes, with no change cost and a success worth v episodes of running cost. Δp : held-out difference in points (Table 1); Δc_{run} : estimated change in running cost per deployed episode, from the tokens of patched agents on solved tasks and of truncated step-limit episodes; C : verification episodes. Intervals combine the uncertainty of Δp and of Δc_{run} (standard error about 0.1), as normal.

every judge (Appendix H). *Replays need controls*: without any patch, 20–23% of replays from the failing step succeed anyway, and the rate ranges from 0 to 0.5 across categories. *Harm is the least measurable quantity*: the mean effect of a patch on previously solved tasks is –1.0 points on retail and –0.6 on airline, with 90% intervals of about ± 2.5 , and patches differ by a spread of 2–3 points; detecting a 3-point regression against a 15% base rate with 90% power takes about 1,200 runs per patch (Proposition 2).

4.4 Is verification worth its cost?

Table 3 evaluates Eq. 1 for every verified set against applying the attributed patches. The ver-

ification bill is at most 10 episodes, about two cents of τ^2 compute at our measured cost, while at $N = 1000$ the 90% intervals of Net(N) span 370–520 episodes at $v = 1$, mostly from the uncertain running cost, and 630–1,830 at $v = 10$. The sign of Net(N) is thus decided by Δp and the running cost, not by the bill, and every interval includes zero. At $v = 10$ the point estimate is negative for every set except CARVE with replays only on retail and CARVE and the net-effect method on airline. Verified sets apply fewer patches, which saves change cost, but even at one episode per patch the saving is at most 6 episodes. In this regime the question “is verification worth its cost” reduces to “does it select better changes”, which Study 1 could

not resolve.

Policies on the banks. Scored on the banks without new runs, the verifying policies were not reliably better than applying the attributed patches either, and edge priors with the audited analyst accuracy only lowered their value (Appendix C).

5 Study 2: Verification Evidence Inside RRSI

Study 1 patches only text, runs one round, draws most failures from injected faults, and compares sets whose predicted differences are below its deployment noise. Study 2 builds on RRSI (Google Cloud AI Research and collaborators, 2026), whose proposer edits harness code over many rounds of natural failures and whose selection step, every candidate evaluated twice on the whole training set, is the most expensive evidence on our list. We run RRSI’s loop, collect every kind of evidence for every candidate it measures, deploy every candidate, and ask what each kind of evidence would have decided. Design, analysis code and power analysis were registered before any held-out episode was run; Appendix M describes the port and every change made after registration.

Loop. We vendor RRSI’s code (Apache 2.0) and change only its model backend, its cost accounting and one rule for empty proposals. Analyst, proposer, critic and judge are DeepSeek-V4-Pro with thinking disabled; the agent and user simulator are DeepSeek-V4-Flash at temperature 0, at which our episode loop matches τ^2 ’s own orchestrator; the starting harness is τ^2 ’s reference agent. Each round the proposer drafts $m = 2$ candidates under an edit budget that anneals from 4 to 1, a critic rejects edits that hard-code task data or touch the grader, and RRSI evaluates the candidates that pass a smoke test twice on every evolve task. It accepts a candidate that beats the incumbent by more than a noise band δ , calibrated from three evaluations of the base harness, or that is within the band and cheaper to run or of an untried kind, and it rejects a harness that raises errors in more than 2% of trials (Appendix M). Changes may touch any part of the harness, including extra model calls, harness-side tools and skill files, but not memory across episodes. Evolve tasks are τ^2 ’s training split (74 retail, 30 airline) and held-out tasks its test split (40 and 20); the loop ran $T = 20$ rounds per domain.

Evidence and decisions. For each measured candidate we collect the evidence of §3 as estimates of its change ΔS in evolve success (Table 19, Appendix M): an LLM judge’s prediction from the diff and the failures it targets (no episodes); *sample@b*, the paired difference on b evolve episodes; *replay@b*, the failure share times the success rate of b targeted replays of failed evolve trials; *replay-null@b*, which subtracts the success rate of null replays with the incumbent; and *net@b*, which splits b between both kinds of replay and regression runs on solved tasks, with or without a Beta prior on the repair rate. Budgets of 10, 40 and 80 episodes per candidate are subsamples of one collection, four replays with each candidate and four null replays of each failed evolve task (at most 20 per round), plus RRSI’s own full evaluation. A replay re-executes the recorded prefix through the new harness and goes live where it would first act differently. Every rule decides with RRSI’s rule, with ΔS replaced by its estimate and δ by twice the estimate’s standard error, so the rules differ only in their evidence.

Ground truth and estimand. Every incumbent and measured candidate is deployed from the first step on the held-out tasks, 240 episodes each (retail 40 tasks $\times$ 6, airline 20 $\times$ 12), and the base and final harnesses on 400 and 240. A candidate’s deployment gain $\Delta p(c | H_t)$ is its held-out success minus that of the incumbent H_t it was proposed against, paired by task. A rule E ’s *decision value* is $D(E) = \frac{1}{40} \sum_t \sum_c P_E(c | t) \Delta p(c | H_t)$, the expected deployment gain of its choice in round t (zero when it keeps H_t), with P_E taken over 400 subsamples of the evidence. D is conditional on RRSI’s trajectory: it scores each rule’s decisions in the states RRSI visited, not a loop driven by that rule, whose later incumbents and candidates would differ. The registered primary endpoints are (P1) *transfer*, the held-out gain of the final harness over the base, and (P2) D at $b = 40$, with three one-sided contrasts, Holm-adjusted at $\alpha = 0.05$: net@40 against full evaluation (non-inferiority, margin 0.75 points per round), net@40 against no verification, and full evaluation against no verification. The bootstrap resamples rounds and held-out tasks within domain; it does not capture variation between trajectories. The registration specified 160 held-out episodes per harness, and more if net@40 and full evaluation disagreed in over 40% of rounds; after six rounds they disagreed

in 58%, so we raised the count to 240 (27% over all 40 rounds; the registered count is a sensitivity analysis).

5.1 Results

The loop. Of 80 drafted candidates, 47 were measured (the critic rejected 21, 9 drafts made no change, 3 failed the smoke test). Thirty-one of the 47 changed harness code, twelve of them in the hook that checks tool calls before they run, and 16 changed only the system prompt or a skill file. RRSI accepted five, two on retail and three on airline, all in the first five rounds; evolve success rose from 86.5% to 90.5% (retail) and from 66.7% to 80.0% (airline). Its noise band was 1.1 points on retail and 17.8 on airline, whose evaluations have 60 trials, so all three airline acceptances were within the band and won on running cost. Most candidates hurt: their mean deployment gain was -1.5 points, and 19 of 47 were positive.

P1: transfer. Retail’s evolve gain of 4.1 points did not transfer: the final harness’s held-out gain was -0.25 points $[-3.00, +2.75]$. Airline’s did: $+8.75$ $[+2.50, +15.83]$ against $+13.3$ on evolve tasks.

P2, as registered. Full evaluation had the highest decision value, $+0.22$ points per round $[-0.06, +0.62]$ (Table 4); sample@40 and sample@80 also had small positive point estimates. Net@40, at a third of the cost, had -0.07 ; the non-inferiority difference was -0.28 $[-0.77, +0.17]$, one-sided $p = 0.058$ (Holm 0.17), so neither non-inferiority nor inferiority was shown. Neither rule was shown better than no verification (full $+2.09$, net@40 $+1.80$; Holm $p = 0.17$). With the registered 160 held-out episodes the non-inferiority difference was -0.48 $[-1.03, +0.01]$, $p = 0.185$. The priors moved net@40 by at most 0.01.

One broken candidate and a common error check. Most of the losses of no verification, the judge and replay without null replays come from one candidate (retail, round 6): a check before item-changing calls that reads an attribute the harness never sets, so the harness raised an error in half of its evolve episodes and held-out success fell by 51.7 points. RRSI’s critic accepted it and the judge predicted a gain of 1.5 points; 6 of its 48 replays still succeeded, which replay@ b counts as repairs, while 15 of 48 null replays succeeded. The registered analysis gave RRSI’s er-

ror check to full evaluation only. To compare evidence rather than error handling, we apply the check to every rule’s own episodes and add two baselines that run ten episodes of each candidate only for the check, then accept a passing candidate at random or by the judge (post hoc; Table 4). Every rule that runs episodes then rejects the broken candidate. Full evaluation still has the highest value; its differences from sample@40, net@40 and replay-null@40 are $+0.18$ $[-0.22, +0.58]$, $+0.26$ $[-0.20, +0.75]$ (non-inferiority $p = 0.049$, Holm 0.15) and $+0.29$ $[-0.06, +0.72]$, and these three differ from each other by at most 0.11. Accepting what passes the execution check loses 0.58 points per round $[-1.94, +0.79]$ (0.43 with the judge), 0.80 $[-0.67, +2.19]$ less than full evaluation: beyond crashes, most candidates still hurt.

Dependence and resolution. Retail’s 20 rounds share 3 incumbents and airline’s 4. Resampling blocks of rounds that share an incumbent widens the standard errors of these contrasts by a factor of 1.3–1.9 (full minus net@40: $[-0.27, +1.00]$); leaving out any one round moves them by at most 0.11 points, and any one incumbent by at most 0.20. With 80% power at a one-sided 5% level, the design detects per-round differences of 0.60–0.74 points between full evaluation and the @40 rules (0.78–1.11 by blocks), 0.29–0.52 among the @40 rules and about 2 against the execution-only baseline; a single candidate’s gain is detected only above about 5.3 points (median standard error 2.1). Apart from the broken candidate, gains were too small to replicate: the split-half correlation of candidates’ held-out gains was 0.76 with it and -0.30 without it. Correlations with deployment gain were 0.85 for RRSI’s evolve difference (0.33 without that candidate), 0.03 (0.08) for the judge and 0.25 (0.26) for replays.

Cost. Full evaluation cost 123 episodes per round and the @40 rules 33–47 episode equivalents, because a replay used about 0.8 of an episode’s tokens. Code changes rarely lengthened replays: 88 of 1,222 replays of code changes went live before the requested step, all for 4 of 31 candidates, and none of 640 replays of text changes did. Against keeping the incumbent, full evaluation’s Net(10^4) at $v = 10$ is $+78$ episodes $[-183, +466]$, positive in 58% of resamples; its break-even horizon of about 6,000 episodes is a point estimate, and at $v = 1$ no rule repays its evidence within 10^4 episodes. The study used about 21,200 episodes

Evidence	b	Registered analysis		Common error check		Accepts	Cost per round episodes / equiv.	Net(10^4) $v = 10$
		D	[90%]	D	[90%]			
None	0	-1.87	[-4.61, +0.25]	-1.87	[-4.61, +0.25]	0.90	0 / 0	-1,904
LLM judge	0	-1.72	[-4.42, +0.28]	-1.72	[-4.42, +0.28]	0.72	0 / 0	-1,763
None + check	10	-	-	-0.58	[-1.94, +0.79]	0.88	12 / 12	-652
Judge + check	10	-	-	-0.43	[-1.61, +0.72]	0.70	12 / 12	-512
Full (RRSI)	all	+0.22	[-0.06, +0.62]	+0.22	[-0.06, +0.62]	0.12	123 / 123	78
Sample	10	-0.34	[-0.99, +0.21]	-0.04	[-0.44, +0.35]	0.30	12 / 12	-69
Replay	10	-0.43	[-1.27, +0.37]	-0.37	[-1.20, +0.45]	0.54	12 / 9	-423
Replay-null	10	-0.05	[-0.26, +0.13]	-0.05	[-0.26, +0.13]	0.08	10 / 8	-69
Net	10	-0.50	[-1.43, +0.23]	-0.07	[-0.57, +0.41]	0.37	11 / 10	-101
Sample	40	+0.04	[-0.38, +0.45]	+0.04	[-0.38, +0.45]	0.28	47 / 47	-6
Replay	40	-1.67	[-4.35, +0.26]	-0.40	[-1.60, +0.78]	0.80	42 / 33	-499
Replay-null	40	-0.07	[-0.35, +0.15]	-0.07	[-0.35, +0.15]	0.10	42 / 34	-125
Net	40	-0.07	[-0.52, +0.36]	-0.04	[-0.50, +0.38]	0.30	44 / 40	-101
Sample	80	+0.08	[-0.35, +0.54]	+0.08	[-0.35, +0.54]	0.26	82 / 82	3
Replay	80	-1.69	[-4.41, +0.27]	-0.40	[-1.59, +0.79]	0.80	47 / 37	-496
Replay-null	80	-0.12	[-0.59, +0.27]	-0.12	[-0.59, +0.27]	0.14	74 / 59	-210
Net	80	-0.02	[-0.49, +0.46]	-0.02	[-0.49, +0.46]	0.27	88 / 81	-119

Table 4: Study 2: decision value D of each rule, the held-out gain of its choice over the incumbent in points per round (keeping the incumbent scores 0), over 40 rounds of both domains, with a 90% bootstrap interval over rounds and tasks. Registered: RRSI’s harness-error check (reject above 2% errors) only inside full evaluation. Common error check (post hoc): the check applied to every rule’s own episodes; + *check* rules run 10 episodes of each candidate only for the check. Accepts, cost and Net(10^4) (Eq. 1 against keeping the incumbent, $N = 10^4$ deployed episodes, a success worth $v = 10$ episodes, in episodes) are for the common check. No candidate was measured in 4 of the 40 rounds. Full: RRSI’s own decision. Best value in bold.

and \$37 (Appendix M).

6 Discussion

What the evidence shows. Our results identify failure modes of verification and the resolution needed to compare it; they do not rank evidence by predictive quality. Most proposed changes did not help held out, so verification earned its place mainly by rejecting harmful ones. Evidence that reads a change without running it, or counts every replay success as a repair, accepted a change that crashed the harness; an execution check caught it, but accepting what passed still lost about half a point per round.

Resolution limited these studies more than price.

A round of full evaluation costs 123 episodes, about 25 cents of τ^2 compute; whether it repays that depends on the horizon and on the value of a success. What limited both studies was the resolution of deployment measurements: here, comparing evidence on ordinary repairs needs per-round differences of 0.6 points or candidate gains above 5 points. Finer comparisons need larger held-out sets than τ^2 ’s test splits, independent trajectories, or loops whose changes matter more.

Implications for self-improvement loops.

Loops that report the final agent’s held-out success (Chen et al., 2026; Zhang et al., 2026b; Google Cloud AI Research and collaborators, 2026) measure all their decisions at once; retail’s evolve gain did not transfer at all. Recording the held-out gain of accepted and rejected changes, with the evidence behind each decision, shows which evidence earns its place, and comparisons of evidence must hold error handling fixed. Replays need null replays, and any evidence that runs the changed agent should reject harness errors.

7 Conclusion

We asked which verification evidence predicts the deployment gain of agent repairs, and whether it is worth its cost. Inside RRSI’s loop most proposed changes lowered held-out success, evidence that reads changes without running them did as badly as no verification, and with a common error check full re-evaluation led but cheaper evidence was neither shown non-inferior nor inferior to it. Ranking evidence on ordinary repairs needs larger held-out sets and independent trajectories.

Limitations

Both studies use one model family, with DeepSeek-V4-Flash as agent and DeepSeek-V4-Pro as analyst, proposer and judge, and two customer-service domains of τ^2 -bench; Study 1 adds ShopDesk. Study 1’s patches rewrite only policy sections, tool descriptions and a step limit; it runs a single round, mostly on injected faults (natural failures are 16–25% of training failures), and deploys patch sets only at budget 10, on 120 (retail) and 60 (airline) held-out episodes, where most of retail’s gain comes from fault-injected episodes. Its bank analyses rest on a harm model fitted to at most 20 regression runs per patch, and its edge priors use an analyst accuracy of 0.7 unless stated. Study 1’s agent used the provider’s default sampling and Study 2’s uses temperature 0. Study 2’s proposer is likely weaker than the model RRSI was developed with; its loop accepted changes only in the first five rounds, and its decision values are dominated by one broken candidate. RRSI’s component labels did not recognise our harness’s files, which may have changed what its proposer was told to explore. Its held-out sets are τ^2 ’s test splits of 40 and 20 tasks, on which the gains of the other candidates were not resolved. Its decision values are conditional on one RRSI trajectory per domain, rounds that share an incumbent are dependent, and our intervals do not capture variation between trajectories. Its registered analysis gave RRSI’s error check to full evaluation only, so the comparison with a common check is post hoc; and its held-out count was raised after six rounds, when some held-out episodes had run and a few results had been seen (Appendix M); the registered count gives the same conclusions. Token prices for the models we used change; we report costs in episodes.

Ethics Statement

Self-improving agents change their own behaviour without a person reviewing each change. In Study 2 most proposed changes lowered held-out success, and one that crashed the harness in half of its episodes passed both an LLM critic and an LLM judge. That Study 1 detected no gain from verification does not show that verification is unnecessary; our measurements cannot exclude small harms per patch and should not be read as licence to skip review. On ShopDesk a single patch silently broke a quarter of tasks (Appendix L). Our environments use synthetic customer data only.

References

- Lakshya A. Agrawal, Shangyin Tan, Dilara Soylu, Noah Ziemis, Rishi Khare, Krista Opsahl-Ong, Arnav Singhvi, Herumb Shandilya, Michael J. Ryan, Meng Jiang, Christopher Potts, Koushik Sen, Alexandros G. Dimakis, Ion Stoica, Dan Klein, Matei Zaharia, and Omar Khattab. 2026. GEPA: Reflective prompt evolution can outperform reinforcement learning. In *International Conference on Learning Representations*.
- Victor Barres, Honghua Dong, Soham Ray, Xujie Si, and Karthik Narasimhan. 2025. τ^2 -bench: Evaluating conversational agents in a dual-control environment. *arXiv preprint arXiv:2506.07982*.
- Akash Bonagiri, Devang Borkar, Gerard Janno Andrias, Setareh Rafatirad, and Houman Homayoun. 2026. CausalFlow: Causal attribution and counterfactual repair for LLM agent failures. *arXiv preprint arXiv:2605.25338*.
- Mert Cemri, Andrei Cojocaru, Melissa Pan, Shu Liu, Shubham Agarwal, Alexander Krentsel, Jay Tang, Kannan Ramchandran, Joseph E. Gonzalez, Matei Zaharia, Alexandros G. Dimakis, and Ion Stoica. 2026. Fantastic adaptive taxonomies and how to use them. *arXiv preprint arXiv:2607.16387*.
- Mert Cemri, Melissa Z. Pan, Shuyi Yang, Lakshya A. Agrawal, Bhavya Chopra, Rishabh Tiwari, Kurt Keutzer, Aditya Parameswaran, Dan Klein, Kannan Ramchandran, Matei Zaharia, Joseph E. Gonzalez, and Ion Stoica. 2025. Why do multi-agent LLM systems fail? *arXiv preprint arXiv:2503.13657*.
- Kathryn Chaloner and Isabella Verdinelli. 1995. Bayesian experimental design: A review. *Statistical Science*, 10(3):273–304.
- Mengzhuo Chen, Junjie Wang, Zhe Liu, Yawen Wang, Haiming Zheng, and Qing Wang. 2026. From failed trajectories to reliable LLM agents: Diagnosing and repairing harness flaws. *arXiv preprint arXiv:2606.06324*.
- Peter I. Frazier, Warren B. Powell, and Savas Dayanik. 2008. A knowledge-gradient policy for sequential information collection. *SIAM Journal on Control and Optimization*, 47(5):2410–2439.
- Google Cloud AI Research and collaborators. 2026. RRSI: Regularized recursive self-improvement of agent harnesses. *arXiv preprint arXiv:2609.24972*.
- Shengran Hu, Cong Lu, and Jeff Clune. 2025. Automated design of agentic systems. In *International Conference on Learning Representations*.
- Omar Khattab, Arnav Singhvi, Paridhi Maheshwari, Zhiyuan Zhang, Keshav Santhanam, Sri Vardhamanan, Saiful Haq, Ashutosh Sharma, Thomas T. Joshi, Hanna Moazam, Heather Miller, Matei Zaharia, and Christopher Potts. 2024. DSPy: Compiling declarative language model calls into self-improving

743	pipelines. In <i>International Conference on Learning Representations</i> .	798
744		799
745	Jiazheng Li, Emine Yilmaz, Bei Chen, and Dieu-	800
746	Thu Le. 2026. Towards self-improving error di-	801
747	agnosis in multi-agent systems. <i>arXiv preprint</i>	802
748	<i>arXiv:2604.17658</i> .	803
749	Xiaofeng Lin, Yingxu Wang, Tung Sum Thomas Kwok,	804
750	Daniel Guo, Sahil Arun Nale, Charles Fleming,	805
751	and Guang Cheng. 2026. REFLECT: Intervention-	
752	supported error attribution for silent failures in LLM	
753	agent traces. <i>arXiv preprint arXiv:2606.09071</i> .	
754	Jiale Liu, Huajun Xi, Shaokun Zhang, Yifan Zeng, Tian-	806
755	wei Yue, Chi Wang, Jian Kang, Qingyun Wu, and	807
756	Huazheng Wang. 2026. Who&when pro: Can LLMs	808
757	really attribute failures in AI agents? <i>arXiv preprint</i>	809
758	<i>arXiv:2607.09996</i> .	810
759	Ming Ma, Jue Zhang, Fangkai Yang, Yu Kang, Qing-	
760	wei Lin, Saravan Rajmohan, and Dongmei Zhang.	
761	2025. DoVer: Intervention-driven auto debug-	
762	ging for LLM multi-agent systems. <i>arXiv preprint</i>	811
763	<i>arXiv:2512.06749</i> .	812
764	Aman Madaan, Niket Tandon, Prakhar Gupta, Skyler	813
765	Hallinan, Luyu Gao, Sarah Wiegrefe, Uri Alon,	
766	Nouha Dziri, Shrimai Prabhumoye, Yiming Yang,	
767	Shashank Gupta, Bodhisattwa Prasad Majumder,	
768	Katherine Hermann, Sean Welleck, Amir Yazdan-	
769	bakhsh, and Peter Clark. 2023. Self-refine: Iterative	
770	refinement with self-feedback. In <i>Advances in Neu-</i>	
771	<i>ral Information Processing Systems</i> .	815
772	Reid Pryzant, Dan Iter, Jerry Li, Yin Tat Lee, Chen-	816
773	guang Zhu, and Michael Zeng. 2023. Automatic	817
774	prompt optimization with “gradient descent” and	818
775	beam search. In <i>Proceedings of the 2023 Confer-</i>	
776	<i>ence on Empirical Methods in Natural Language</i>	
777	<i>Processing</i> .	
778	Jaineet Shah. 2026. Causal agent replay: Counterfactual	
779	attribution for LLM-agent failures. <i>arXiv preprint</i>	
780	<i>arXiv:2606.08275</i> .	
781	Noah Shinn, Federico Cassano, Ashwin Gopinath,	
782	Karthik Narasimhan, and Shunyu Yao. 2023. Re-	
783	flexion: Language agents with verbal reinforcement	
784	learning. In <i>Advances in Neural Information Pro-</i>	
785	<i>cessing Systems</i> .	
786	Edward K. Smith, Earl T. Barr, Claire Le Goues, and	
787	Yuriy Brun. 2015. Is the cure worse than the disease?	
788	overfitting in automated program repair. In <i>Proceed-</i>	
789	<i>ings of the 10th Joint Meeting on Foundations of</i>	
790	<i>Software Engineering</i> .	
791	Rakshith Vasudev, Melisa Russak, Dan Bikel, and	
792	Waseem Alshikh. 2026. Accurate failure prediction	
793	in agents does not imply effective failure prevention.	
794	<i>arXiv preprint arXiv:2602.03338</i> .	
795	Abraham Wald. 1945. Sequential tests of statistical	
796	hypotheses. <i>The Annals of Mathematical Statistics</i> ,	
797	16(2):117–186.	
	Mengting Wan, Tara Safavi, Sujay Kumar Jauhar, Yujin	798
	Kim, Scott Counts, Jennifer Neville, Siddharth Suri,	799
	Chirag Shah, Ryen W. White, Longqi Yang, Reid	800
	Andersen, Georg Buscher, Dhruv Joshi, and Nagu	801
	Rangan. 2024. TnT-LLM: Text mining at scale with	802
	large language models. In <i>Proceedings of the 30th</i>	803
	<i>ACM SIGKDD Conference on Knowledge Discovery</i>	804
	<i>and Data Mining</i> .	805
	Xiaojun Wu, Cehao Yang, Honghao Liu, Xueyuan Lin,	806
	Wenjie Zhang, Zhichao Shi, Xuhui Jiang, Chengjin	807
	Xu, Jia Li, and Jian Guo. 2026. Bayesian-Agent:	808
	Posterior-guided skill evolution across LLM agent	809
	harnesses. <i>arXiv preprint arXiv:2606.08348</i> .	810
	Chengrun Yang, Xuezhi Wang, Yifeng Lu, Hanxiao Liu,	
	Quoc V. Le, Denny Zhou, and Xinyun Chen. 2024.	
	Large language models as optimizers. In <i>Interna-</i>	
	<i>tional Conference on Learning Representations</i> .	811
	Shunyu Yao, Noah Shinn, Pedram Razavi, and Karthik	812
	Narasimhan. 2024. τ -bench: A benchmark for tool-	813
	agent-user interaction in real-world domains. <i>arXiv</i>	814
	<i>preprint arXiv:2406.12045</i> .	
	Mert Yuksekogonul, Federico Bianchi, Joseph Boen,	815
	Sheng Liu, Pan Lu, Zhi Huang, Carlos Guestrin, and	816
	James Zou. 2025. Optimizing generative AI by back-	817
	propagating language model feedback. <i>Nature</i> , 639.	818
	Guibin Zhang, Junhao Wang, Junjie Chen, Wangchun-	819
	shu Zhou, Kun Wang, and Shuicheng Yan. 2026a.	820
	AgentTracer: Who is inducing failure in the LLM	821
	agentic systems? In <i>International Conference on</i>	822
	<i>Learning Representations</i> . ArXiv:2509.03312.	
	Jenny Zhang, Shengran Hu, Cong Lu, Robert Lange,	823
	and Jeff Clune. 2025a. Darwin Gödel machine:	824
	Open-ended evolution of self-improving agents.	825
	<i>arXiv preprint arXiv:2505.22954</i> .	826
	Qizheng Zhang, Changran Hu, Shubhangi Upasani,	827
	Boyuan Ma, Fenglu Hong, Vamsidhar Kamanuru,	
	Jay Rainton, Chen Wu, Mengmeng Ji, Hanchen Li,	
	Urmish Thakker, James Zou, and Kunle Olukotun.	
	2026b. Agentic context engineering: Evolving con-	
	texts for self-improving language models. In <i>Inter-</i>	
	<i>national Conference on Learning Representations</i> .	828
	Shaokun Zhang, Ming Yin, Jieyu Zhang, Jiale Liu,	829
	Zhiguang Han, Jingyang Zhang, Beibin Li, Chi	830
	Wang, Huazheng Wang, Yiran Chen, and Qingyun	831
	Wu. 2025b. Which agent causes task failures	
	and when? on automated failure attribution of	
	LLM multi-agent systems. In <i>Proceedings of the</i>	
	<i>42nd International Conference on Machine Learning</i> .	
	ArXiv:2505.00212.	832
	Andrew Zhao, Daniel Huang, Quentin Xu, Matthieu	833
	Lin, Yong-Jin Liu, and Gao Huang. 2024. ExpeL:	834
	LLM agents are experiential learners. In <i>Proceedings</i>	835
	<i>of the AAAI Conference on Artificial Intelligence</i> .	836
	Kunlun Zhu, Zijia Liu, Bingxuan Li, Muxin Tian, Yingx-	837
	uan Yang, Jiaxun Zhang, Pengrui Han, Qipeng Xie,	838
	Fuyang Cui, Weijia Zhang, Xiaoteng Ma, Xiaodong	839
		840
		841
		842
		843
		844
		845
		846
		847
		848
		849
		850
		851
		852
		853

Yu, Gowtham Ramesh, Jialian Wu, Zicheng Liu, Pan Lu, James Zou, and Jiaxuan You. 2025. Where LLM agents fail and how they can learn from failures. *arXiv preprint arXiv:2509.25370*.

Kunlun Zhu, Xuyan Ye, Zhiguang Han, Yuchen Zhao, Bingxuan Li, Weijia Zhang, Muxin Tian, Xiangru Tang, Pan Lu, James Zou, Jiaxuan You, and Heng Ji. 2026. AgentDebugX: An open-source toolkit for failure observability, attribution, and recovery in LLM agents. *arXiv preprint arXiv:2607.18754*.

A An Accounting Framework for Single-Round Repair

Setting. An agent is assembled from *components* j (prompt sections, tool descriptions, configuration). On a task distribution it fails with rate F ; an LLM groups its failures into categories i with shares f_i . For each *cell* (i, j) that the analyst blames, it proposes K candidate patches k that rewrite component j . Let $e_{ij} \in \{0, 1\}$ indicate whether changing j can repair category i at all, and q_k the fraction of category- i failures that patch k repairs if so. Separating e from q matters because a patch that fails to help may be aimed at the wrong component or merely be a bad patch. We call a cell with $e_{ij} = 1$ an *effective edge*. This is an interventional notion: some change to j repairs failures of category i . It need not mean that j was the original cause of those failures.

Net effect. Applying patch k of cell (i, j) changes the success rate by

$$\Delta_k = F f_i e_{ij} q_k - (1 - F) \rho_k - c_a - c_w (1 - e_{ij}), \quad (2)$$

where ρ_k is the patch’s absolute excess failure rate on tasks that used to succeed (regressions, which may hit any category): such a task fails a re-run with probability r_0 unpatched and $r_0 + \rho_k$ patched, so $\rho_k < 0$ is possible; $c_a \geq 0$ is a cost charged for every applied change (review, maintenance) and $c_w \geq 0$ a penalty charged only for a change aimed at a wrong component (risk not captured by ρ_k). Costs are fractions of the failure mass F . The goal is to choose a set of patches, at most one per cell, that maximises $\sum_k \Delta_k$ on held-out tasks; effects of patches to different components are treated as additive, and held-out success of whole patch sets (Table 1) and a leave-one-out ablation (Appendix J) are the only checks. A targeted replay of a category- i failure with patch k succeeds with probability $b_i + (1 - b_i) e_{ij} q_k$, a null replay with probability b_i , and a regression run fails with probability $r_0 + \rho_k$.

Value of verification. Before any evidence, the net effects $\Delta = (\Delta_k)$ of the candidate patches are uncertain, with a prior that reflects the analyst’s accuracy and what is known about patch quality and regressions. The *default* applies, in each cell, the patch with the largest $\mathbb{E}[\Delta_k]$ if that is positive. Verification observes outcomes Y and then chooses; its value is $V(Y) = \mathbb{E}[\max_S \mathbb{E}[\sum_{k \in S} \Delta_k \mid Y]] - \max_S \mathbb{E}[\sum_{k \in S} \Delta_k]$, before the cost of the experiments. Write $x^+ = \max(x, 0)$ and $x^- = \max(-x, 0)$.

Proposition 1 (Value of verification). *For any experiments Y , $V(Y)$ is at most the value of learning Δ exactly, which is the sum over cells of*

$$V_{ij} = \mathbb{E}[\max_k \Delta_k^+] - \max_k (\mathbb{E} \Delta_k)^+. \quad (3)$$

With k^* the default patch of the cell,

$$V_{ij} \leq \underbrace{\mathbb{E}[\max_k \Delta_k^+ - \Delta_{k^*}^+]}_{\text{choosing among patches}} + \underbrace{\mathbb{E}[\Delta_{k^*}^-]}_{\text{avoiding harm}}, \quad (4)$$

with equality when the default applies k^* , and $\mathbb{E}[\Delta_{k^*}^-] \leq (1 - F) \mathbb{E}[\rho_{k^*}^+] + c_a + c_w P(e_{ij}=0)$.

Corollary 1 (Edge-only verification). *Suppose a cell has a single candidate patch whose net effect depends only on e_{ij} : $\Delta = G$ if $e_{ij} = 1$ and $\Delta = -H$ if $e_{ij} = 0$, with $G, H \geq 0$ known. With $\pi = P(e_{ij}=1)$, $V_{ij} = \min(\pi G, (1 - \pi)H)$.*

What survives deployment constraints. Deployment may use a heuristic default D , applies at most one patch per component, which couples cells, combines repairs of one category as $1 - \prod_k (1 - q_k)$ (overstated by at most $f_i \sum_{k < l} q_k q_l$ when added), and cannot push the failure rate on solved tasks below zero ($-\sum_k \rho_k \leq r_0$). Three statements survive. (i) The first sentence of the proposition holds for any set of admissible actions, so $V(Y)$ is at most the value of perfect information over deployable sets. (ii) Over a heuristic default that value is $V + G_D$, where $G_D = \max_S \mathbb{E}[\sum_{k \in S} \Delta_k] - \mathbb{E}[\sum_{k \in D} \Delta_k] \geq 0$ is the *decision-rule gap* that better use of the prior alone would close. (iii) The value of dropping harmful default patches is at most their expected harm, $\sum_{k \in D} \mathbb{E}[(1 - F) \rho_k^+ + c_a + c_w (1 - e_{ij})]$, because dropping a patch from an overlapping set loses at most its marginal repair. What does not survive is the separation over cells: with one patch per component, the choice and discovery terms have no closed form, and we compute them in sequence (harm,

then choice on the default’s components, then any component) with a greedy optimiser, which gives a lower bound on the constrained value of perfect information under a given harm model. The three sources of value are *avoiding harm*, which needs regressions or change costs because replay success alone cannot create it; *choosing among patches* of different quality; and *discovery* of edges the analyst missed, the whole term $\mathbb{E}[\max_k \Delta_k^+]$ in cells the default leaves alone. The corollary fails when quality or regressions are uncertain given the edge ($\pi = 1$ with $\Delta = +0.10$ or -0.05 equally likely gives $V = 0.025$).

Proposition 2 (Cost of evidence). *Under Wald’s approximations, a sequential test that decides e_{ij} from targeted replays with error δ needs on average about $\log \frac{1-\delta}{\delta} / \text{KL}(b + (1-b)q \parallel b)$ replays when the edge is real. Detecting a regression of size ρ against a base rate r_0 with a one-sided level- α test and power $1 - \beta$ needs about $(z_\alpha + z_\beta)^2 r_0 (1 - r_0) / \rho^2$ regression runs.*

Two consequences follow. First, targeted replays are uninformative about the harm term, because they only see the target category: a patch can pass every replay and still break other tasks. Second, small regressions are expensive to see. With $r_0 = 0.1$, a regression of $\rho = 0.25$ is detected with 90% power by 20 runs, whereas $\rho = 0.03$ needs about 850. Proposition 1 is the standard value-of-information argument (Chaloner and Verdinelli, 1995); what is new is its decomposition for patches and its evaluation on measured banks.

Where could verification add value? Table 5 evaluates Proposition 1 on deployable sets. Every value in it is conditional on a model of the harm ρ_k that the regression runs cannot pin down per patch, so we read only its qualitative pattern. *Defaults.* The Bayes default, built from the net-effect method’s priors, scores 1.49 oracles on retail against 1.43 for applying the attributed patches, and 0.53 against 0.66 on airline; $G_D \geq 0$ holds only under the prior, so the negative realised gap on airline means the edge prior is too generous there. *Value.* Under the binomial model, perfect information is worth 1.18 oracles over the heuristic on retail (90% interval $[0.10, 2.99]$, about 8 points of absolute success) and 0.86 on airline ($[0.24, 1.55]$, about 10 points), split into 0.41, 0.57 and 0.20 (retail) and 0.20, 0.36 and 0.29 (airline) for harm, choice and discovery. Under the earlier normal models the retail value ranges from 0.87 (complete pooling) to

	Default rule	Value	Gap	Harm	Choice	Discov.	Total
Retail	Apply, $c = 0$	1.43	0.06	0.41	0.57	0.20	1.18
	Bayes, $c = 0$	1.49	–	0.80	0.32	0.00	1.12
	Apply, $c_w = 0.02$	1.10	–0.19	0.54	0.56	0.03	1.13
	normal, partial pooling	0.77		0.87	0.98	0.17	2.02
	normal, $\tau=0$	1.12		0.23	0.22	0.42	0.87
	no pooling	–1.82		3.30	1.21	0.13	4.65
	tasks resampled	–0.52		2.28	0.90	0.13	3.31
	per cell	0.64		1.39	0.60	0.16	2.15
Airline	Apply, $c = 0$	0.66	–0.13	0.20	0.36	0.29	0.86
	Bayes, $c = 0$	0.53	–	0.55	0.44	0.00	0.98
	Apply, $c_w = 0.02$	0.54	0.02	0.27	0.36	0.11	0.74
	normal, partial pooling	0.73		0.04	0.25	0.40	0.69
	normal, $\tau=0$	0.73		0.04	0.25	0.40	0.69
	no pooling	0.39		0.91	0.10	0.15	1.15
	tasks resampled	0.63		0.39	0.25	0.23	0.87
	per cell	0.86		0.15	0.11	0.30	0.56

Table 5: Value of perfect verification over two no-evidence defaults, deployable sets, posterior means under the task-level binomial harm model (Appendix I) as fractions of the repair-only oracle. *Gap*: Bayes default minus Apply attributed (G_D with the sign reversed; negative when the heuristic does better). Harm, choice and discovery are added in that order: dropping default patches with negative net effect, then the best cell and patch on the default’s components, then any other component. Indented rows give the value over Apply at $c = 0$ under the normal-approximation harm models of an earlier analysis (partial, complete and no pooling, tasks resampled before fitting) and with the additive per-cell form of Proposition 1 instead of deployable sets. A value of 1 is 0.070 (retail) or 0.114 (airline) in absolute success.

4.65 (no pooling) and airline’s from 0.69 to 1.15; dropping the support constraint, so that the uncertain benefits of every unblamed patch add up without limit, gives 9.5 on retail (Table 14). Only coarse features are stable: all three terms contribute, and most of the value needs evidence about patches the default applies (harm and choice) rather than discovery of missed edges. Proposition 1 names the quantities that could make verification pay, but it cannot decide whether to verify a new diagnosis: its most influential input, each patch’s harm, takes hundreds of regression runs to measure (Proposition 2), and its values move several-fold with the harm model.

B Proofs

Proposition 1. For any signal Y and any action set, $\max_S \mathbb{E}[U(S) \mid Y] \leq \mathbb{E}[\max_S U(S) \mid Y]$, so taking expectations gives $V(Y) \leq \mathbb{E}[\max_S U(S)] - \max_S \mathbb{E}[U(S)]$, the value of perfect information; this is the standard argument and needs no assumption on how Y relates to Δ . With $U(S) = \sum_{k \in S} \Delta_k$ and at most one patch per cell, both maxima separate over cells: $\max_S U(S) = \sum_{ij} \max_k \Delta_k^+$

and $\max_S \mathbb{E}[U(S)] = \sum_{ij} \max_k (\mathbb{E}\Delta_k)^+$, which gives V_{ij} ; no independence between cells is needed. For Eq. 4, let k^* maximise $\mathbb{E}\Delta_k$. If $\mathbb{E}\Delta_{k^*} \geq 0$, then $V_{ij} = \mathbb{E}[\max_k \Delta_k^+] - \mathbb{E}[\Delta_{k^*}^+] + \mathbb{E}[\Delta_{k^*}^-]$, which is Eq. 4 with equality. Otherwise $V_{ij} = \mathbb{E}[\max_k \Delta_k^+] = \mathbb{E}[\max_k \Delta_k^+ - \Delta_{k^*}^+] + \mathbb{E}[\Delta_{k^*}^+]$, and $\mathbb{E}[\Delta_{k^*}^+] < \mathbb{E}[\Delta_{k^*}^-]$ because $\mathbb{E}\Delta_{k^*} < 0$. Finally, the repair term $F f_{ij} q_k$ is non-negative, so $\Delta_k^- \leq (1 - F)\rho_k^+ + c_a + c_w(1 - e_{ij})$; take expectations.

Heuristic defaults and constraints. For a default D , $\mathbb{E}[\max_S U(S)] - \mathbb{E}[U(D)] = (\mathbb{E}[\max_S U(S)] - \max_S \mathbb{E}[U(S)]) + (\max_S \mathbb{E}[U(S)] - \mathbb{E}[U(D)])$, and the second bracket is $G_D \geq 0$ by definition of the maximum. Restricting S to deployable sets leaves the argument unchanged. For repairs in one category, the Bonferroni inequality gives $1 - \prod_k (1 - q_k) \geq \sum_k q_k - \sum_{k < l} q_k q_l$, which bounds the overstatement of the additive form. The bound on $-\sum_k \rho_k$ holds because the patched failure rate $r_0 + \sum_k \rho_k$ on solved tasks is non-negative under additivity. For (iii), removing patch k from a set S changes its value by $-\text{rep}(S) + \text{rep}(S \setminus k) + (1 - F)[\bar{\rho}(S) - \bar{\rho}(S \setminus k)] + c_a + c_w(1 - e_{ij})$, where rep is the product-form repair and $\bar{\rho}$ the capped sum of harms. Repair is monotone in S , so the first bracket is non-negative, and capping $\sum \rho$ from below changes it by at most ρ_k^+ when k is removed; summing over the dropped patches and taking expectations gives the bound. The greedy optimiser keeps, per component, the cell and patch with the largest positive net effect, which is a feasible deployable set, so the value it attains is a lower bound on the constrained optimum.

Corollary 1. With one patch, $V = \mathbb{E}[\Delta^+] - (\mathbb{E}\Delta)^+ = \pi G - (\pi G - (1 - \pi)H)^+ = \min(\pi G, (1 - \pi)H)$. The assumption that Δ is a function of e_{ij} is what makes the edge state sufficient: if q_k or ρ_k is uncertain given e_{ij} , learning only e_{ij} leaves value on the table and the general expression applies.

Proposition 2. The first part is Wald’s approximation for a sequential probability ratio test between $p_1 = b + (1 - b)q$ and $p_0 = b$: with thresholds $\pm \log \frac{1 - \delta}{\delta}$, neglecting overshoot, the expected sample size under p_1 is the threshold divided by the per-observation drift $\text{KL}(p_1 \| p_0)$. The second is the normal-approximation sample size of a one-sided test of a binomial proportion, evaluated with the

variance at r_0 .

C Verification Policies on the Intervention Banks

Summary. The banks give the same answer as the deployment comparison without new runs. In that comparison the policy seed added no detectable variance: the variance of the seed means of the verified sets’ differences from Apply attributed is 0.0007, against 0.0019 expected from run-to-run noise alone. Under the task-level harm model, applying the attributed patches is worth 1.43 (retail) and 0.66 (airline) of the repair-only oracle, and the verifying policies 0.39–1.56 at budget 40, beating it in 19–57% of posterior draws; scored on bank halves that share no trajectory with the policies’ evidence, they beat it in 22–47%. Policies that use the analyst’s blame as an edge prior used an analyst accuracy of 0.7; with the audited 0.41 and 0.29, or half of these, every such policy is worth less than with 0.7, on both domains (Table 8).

Policies. *Net-effect allocation.* A spike-and-slab model per cell takes its edge prior from the attribution counts (Eq. 7, Appendix O), with grid posteriors for q_k and Beta posteriors for each b_i and each patch’s regression failure rate r_k . It estimates Eq. 2 as $P(e_{ij}=1 \mid \mathcal{D}) F f_{ij} \mathbb{E}[q_k \mid \mathcal{D}] - (1 - F) \max(0, \mathbb{E}[r_k \mid \mathcal{D}] - r_0) / (1 - r_0) - c$, a stricter objective than Eq. 2: it charges c on every applied change, never credits a patch for lowering failures on solved tasks, and scales harm by $1/(1 - r_0)$; Table 12 ablates the last two. It repeatedly runs the targeted replay, null replay or regression run with the largest n -step knowledge gradient per unit cost, $n \in \{1, 2, 4, 8\}$ (Frazier et al., 2008), and applies every patch with positive posterior net effect. *Apply attributed* uses no evidence: for each category it applies the first patch of the most-blamed component and of any component with at least 40% of the blame. *Replay-each* tests hypotheses in order of blame with three replays and accepts on two successes, as in do-then-verify methods (Ma et al., 2025). *HarnessFix-style, per patch* adds two regression runs that must break nothing; these counts are ours. *HarnessFix-style, bundle gate* is closer to the original, which validates a whole bundle of repairs on held-out tasks and promotes it if it solves at least one more task and breaks at most two (Chen et al., 2026); we apply the *Apply attributed* bundle, draw validation episodes from the bank, and allow three attempts. *Uncertainty sampling* replays the

cell with the largest frequency-weighted posterior uncertainty, with or without three cheap checks per patch first. CARVE and its variants are described in Appendix O.

Scoring. Policies run offline by drawing recorded outcomes from the bank, with replacement (without, as a check), with costs in measured tokens and a full replay as the unit; resampling seeds measure policy variability given the bank, not new evidence. *Adjudication:* a cell is an effective edge if its best patch’s replay success exceeds the category’s null rate by at least 0.2 (one-sided Fisher $p < 0.05$), giving 4 effective edges on retail and 7 on airline; choosing the best of three patches on the same replays makes this label optimistic, so we also score policies on bank halves they did not see. *Harm:* every patch’s ρ_k is drawn from the task-level binomial model of Appendix I, and edges and repair rates from a bootstrap of the replays. *Deployable sets:* every accepted set is reduced to one patch per component, keeping the cell of the most frequent category, as on held-out tasks. Values are fractions of the *repair-only oracle*, the failure mass repaired by the effective edges’ best patches with no cost, a fixed yardstick worth 0.070 (retail) and 0.114 (airline) in absolute success. The unpatched baseline (10 runs on each of 111 regression tasks) was collected about 12 hours after the regression runs; unpatched held-out success did not change over that time (0.816 vs. 0.817 on clean retail episodes). *Protocol:* retail was the development domain for every method, including the variants in Appendix K; airline was evaluated once for the original policies and once, later, for the net-effect method, which was frozen on retail beforehand.

Results. Table 6 reports each policy’s posterior expected value. On retail, applying the attributed patches is worth 1.43 oracles (10.1 points of success) and is the best policy at budget 10; at 40 and 80, CARVE with a robust check is ahead (1.56 and 1.59) but beats applying everything in only 57% of draws at budget 40; across budgets, the other verifying policies are worth 0.55–1.49. On airline, applying everything scores 0.66 and no verifying policy exceeds 0.73. The 90% intervals span about three oracles on retail (applying everything: $[-0.52, 2.67]$) and one and a half on airline. On source-split halves every value falls, because each half supports fewer effective edges: applying everything scores 0.96 (retail) and 0.23 (airline), and verifying policies beat it in 22–42% (retail)

and 35–47% (airline) of draws at budget 40. With $c_w = 0.02$ per wrong change (Table 7) applying everything falls to 1.10 and 0.54 on the full banks; the leaders change with budget and split, and none leads everywhere. On the banks, as on held-out tasks, no verification policy reliably beats applying the attributed patches.

Priors. Every policy with an edge prior used an analyst accuracy of $\hat{r} = 0.7$, well above the audited 0.41 (retail) and 0.29 (airline). Table 8 reruns their budget-40 decisions with the audited accuracy and with half of it. Every such policy is worth less than with 0.7 on both domains, by 0.11 to 0.65 oracles on retail and 0.04 to 0.25 on airline at the audited accuracy, and halving it again changes values by at most 0.30 more. With the audited or the weaker accuracy, no verifying policy is worth more than applying the attributed patches.

D Significance-Thresholded Scoring

Charging only regressions significant against the paired-trial r_0 reproduces the picture of a scoring that sets uncertain harm to zero: applying everything recovers 84% and 60% of the oracle and no verifying policy reaches it. With $c_w = 0.02$ per wrong change, the per-patch HarnessFix-style rule (retail, budgets 10–40; airline, 40) and the net-effect method (retail, 80; airline, 80, tied with uncertainty sampling with checks) lead, and CARVE declines with budget on airline (0.30→0.19). Appendix F varies c for both cost types; the winner changes with c .

E Robustness at Budget 40

F Change-Cost Sweep

Table 11 varies the change cost. The best policy changes with c : applying the attributed patches is best on retail at $c = 0$ (on airline, CARVE told $c = 0$ edges it, 0.71 against 0.60), replay-each at $c = 0.01$ on retail, the per-patch HarnessFix-style rule from $c = 0.01$ to 0.02 on airline and at 0.02 on retail, and the net-effect method at $c = 0.05$ on retail under both cost types and on airline per applied change. Charging every applied change rather than only wrong ones lowers every policy but changes the winner in only one setting (airline, $c = 0.05$). Told $c = 0$, the policies that use a cost internally accept patches with significant regressions, which is why some of them are negative in the first column.

Budget (episodes)	τ^2 retail (development)						τ^2 airline					
	full bank			source-split halves			full bank			source-split halves		
	10	40	80	10	40	80	10	40	80	10	40	80
Apply attributed	1.43	1.43	1.43	0.96	0.96	0.96	0.66	0.66	0.66	0.23	0.23	0.23
Bayes default (Prop. 1)	1.49	1.49	1.49	—	—	—	0.53	0.53	0.53	—	—	—
Replay-each	1.37	1.26	1.35	0.85	0.81	0.83	0.60	0.53	0.54	0.18	0.15	0.11
HarnessFix-style, per patch	0.55	1.01	1.28	0.55	0.87	0.97	0.19	0.45	0.53	0.09	0.24	0.22
HarnessFix-style, bundle	0.68	1.16	1.31	0.38	0.56	0.67	0.36	0.57	0.60	0.12	0.17	0.17
Uncertainty sampling	1.27	1.39	1.32	0.84	0.94	0.94	0.59	0.65	0.67	0.10	0.22	0.24
Uncertainty + checks	1.36	1.33	1.35	0.73	0.87	0.98	0.54	0.58	0.65	0.04	0.14	0.18
CARVE, replay only	1.23	1.41	1.49	0.87	0.91	0.98	0.71	0.65	0.70	0.24	0.24	0.24
CARVE	1.30	1.12	0.93	0.61	0.54	0.63	0.63	0.39	0.25	0.11	0.08	0.09
CARVE, measured check	1.23	1.41	1.49	0.82	0.93	1.01	0.71	0.65	0.71	0.24	0.23	0.23
CARVE, robust check	1.35	1.56	1.59	0.84	0.96	1.02	0.68	0.64	0.73	0.26	0.24	0.23
Net-effect (ours)	1.02	1.09	1.29	0.82	0.87	0.97	0.54	0.60	0.64	0.14	0.18	0.19

Table 6: Posterior expected value of each policy’s deployable patch set on the τ^2 banks at $c = 0$, under the task-level binomial harm model, as a fraction of the repair-only oracle; bold is the best per column. *Full bank*: policies and scoring use the same bank; 300 posterior draws, each also bootstrapping the replays, averaged over 40 policy seeds. *Source-split halves*: the bank is split into halves that share no source trajectory and no regression task; policies run on one half, with their inputs (experiment costs, spurious recovery, judge calibration) from that half and the audited analyst accuracy, and are scored on the other (5 halvings $\times$ 100 draws, 20 seeds). The Bayes default is Proposition 1’s no-evidence rule under the net-effect method’s priors. Values above 1 arise when patches are estimated to reduce failures on previously solved tasks.

G Policy Inputs and Regression Sampling

Attribution, taxonomy induction, labelling and candidate patches are shared by all policies and not charged to any budget. Every policy that uses an edge prior uses Eq. 7 with analyst accuracy $\hat{r} = 0.7$, not the audited 0.41 and 0.29 of Table 2; Table 8 rescores them with the audited accuracy and half of it. Three further inputs come from the bank itself and are also uncharged: the cost of each kind of experiment (mean tokens), the mean spurious recovery $\hat{b}$, which CARVE and uncertainty sampling plug in and the net-effect method uses only as a prior mean for its per-category b_i , and, for the two CARVE variants that use them, the judge’s measured sensitivity and false-positive rate. The re-run failure rate r_0 comes from paired re-runs of training tasks. *Regression tasks* are drawn uniformly, with replacement, from the clean training episodes the agent solved, so they follow the training-task distribution of the same domain, not the held-out tasks, which also include fault-injected episodes. The extension to 20 runs covered every patch an original policy accepted; 98–100% of the patches that any policy, including the later net-effect method, deploys at budget 40 have 20 runs, and a policy asking for more runs of a 4-run patch resamples those 4.

H Judge Configurations

The three new configurations regenerate the failing step once per pair and judge it with each prompt; the short prompt shows the task, the diagnosed root cause, the tool results or user message just before the failing step, and the old and new actions. Pairs are stratified on the paired replay’s outcome; because a check and its paired replay are separate regenerations, the check is independent of that outcome given the patch, so the stratification does not bias the patch-level fit. The short-prompt judges agree with the bank judge on 64–76% of pairs. The experiment used about 310 episode-equivalents of tokens. *Why the two definitions differ*. Conditioning on a single replay’s outcome attenuates both quantities towards the judge’s overall pass rate, because many failed replays come from patches that do repair some failures and many successful ones from spurious recovery; the patch-level fit removes this noise, which raises the bank judge’s break-even ratio from about 0.1 to 0.25 (retail) and 0.29 (airline). Across categories, using each category’s own spurious recovery and pairs (≥ 20), the single-replay ratio of the bank judge ranges from 0 to 2.0 on retail and 0.04 to 1.5 on airline. *Latency* was not measured; counted in sequential model calls, a check is two (regeneration and judge) and a replay from the failing step a median of 6 (retail) or 7 (airline), interquartile range 4–11. *Money*: we cannot

Budget (episodes)	τ^2 retail (development)						τ^2 airline					
	full bank			source-split halves			full bank			source-split halves		
	10	40	80	10	40	80	10	40	80	10	40	80
Apply attributed	1.10	1.10	1.10	0.61	0.61	0.61	0.54	0.54	0.54	0.06	0.06	0.06
Bayes default (Prop. 1)	0.91	0.91	0.91	–	–	–	0.56	0.56	0.56	–	–	–
Replay-each	1.10	1.03	1.13	0.54	0.52	0.51	0.49	0.45	0.44	0.02	0.01	–0.06
HarnessFix-style, per patch	0.55	0.93	1.10	0.55	0.80	0.78	0.18	0.42	0.48	0.06	0.18	0.10
HarnessFix-style, bundle	0.52	0.89	1.02	0.22	0.30	0.40	0.29	0.46	0.48	0.02	0.01	0.02
Uncertainty sampling	1.00	1.20	1.13	0.79	0.82	0.74	0.53	0.59	0.61	0.03	0.13	0.12
Uncertainty + checks	1.10	1.19	1.21	0.69	0.79	0.85	0.47	0.53	0.60	–0.02	0.06	0.07
CARVE, replay only	0.92	1.14	1.15	0.75	0.71	0.67	0.62	0.56	0.60	0.13	0.11	0.07
CARVE	1.08	1.02	0.89	0.56	0.44	0.49	0.55	0.33	0.21	0.04	–0.02	–0.03
CARVE, measured check	0.92	1.14	1.15	0.70	0.73	0.69	0.63	0.56	0.62	0.14	0.10	0.06
CARVE, robust check	1.04	1.27	1.25	0.71	0.75	0.70	0.61	0.56	0.64	0.16	0.11	0.06
Net-effect (ours)	0.87	0.98	1.15	0.77	0.78	0.83	0.49	0.55	0.58	0.08	0.08	0.06

Table 7: As Table 6, with a cost of $c_w = 0.02$ of the failure mass charged per patch aimed at a wrong edge.

	Retail			Airline		
	0.7	0.41	0.20	0.7	0.29	0.15
Analyst accuracy $\hat{r}$	0.7	0.41	0.20	0.7	0.29	0.15
Apply attributed	1.43	1.43	1.43	0.66	0.66	0.66
Uncertainty sampling	1.39	1.02	0.95	0.65	0.40	0.35
Uncertainty + checks	1.33	0.99	1.00	0.58	0.35	0.26
CARVE, replay only	1.41	1.22	1.10	0.65	0.57	0.49
CARVE	1.12	0.47	0.17	0.39	0.17	0.12
CARVE, measured check	1.41	1.22	1.10	0.65	0.61	0.50
CARVE, robust check	1.56	1.27	1.14	0.64	0.53	0.47
Net-effect	1.09	0.98	0.99	0.60	0.42	0.40

Table 8: Policies that use the edge prior of Eq. 7, with the analyst accuracy used in the main analysis (0.7), the audited accuracy and half of it; posterior value of deployable sets at budget 40 and $c = 0$ on the full bank (binomial harm model). Apply attributed uses no prior.

state prices for the models we used that will remain accurate, so the table reports token cost with the analyst model priced at one, two and four times the agent model.

I Harm Model, Checks and Sensitivity

Model. Regression outcomes are modelled per patch and task, together with the unpatched baseline runs on the same tasks. With n_{0t} unpatched runs of regression task t and f_{0t} failures, and n_{kt} runs of patch k on task t with f_{kt} failures,

$$f_{0t} \sim \text{Bin}(n_{0t}, \sigma(\alpha_t)),$$

$$f_{kt} \sim \text{Bin}(n_{kt}, \sigma(\alpha_t + \beta(\alpha_t - \bar{\alpha}) + \delta_k)),$$

with $\alpha_t \sim N(a, s_a^2)$, $\delta_k \sim N(\mu, s_d^2)$, $a, \mu \sim N(0, 3^2)$, $s_a \sim \text{HalfNormal}(2)$, $s_d \sim \text{HalfNormal}(1)$, $\beta \sim N(0, 1)$, and $\bar{\alpha}$ the logit of the mean unpatched failure rate. Task effects are shared by the baseline and every patch, so tasks that often fail unpatched do not make patches look

harmful, and the common slope β lets patches move every task’s failure rate towards or away from the middle. A patch’s harm is its excess failure probability averaged over the regression tasks, weighted by how often each task was solved among clean training episodes,

$$\rho_k = \sum_t w_t [\sigma(\alpha_t + \beta(\alpha_t - \bar{\alpha}) + \delta_k) - \sigma(\alpha_t)],$$

which respects the support $\rho_k \geq -r_0$ by construction; patches without runs draw δ_k from the population. We sample by Metropolis within Gibbs, four chains, 2,400 retained draws; $\hat{R} \leq 1.006$ for μ , s_d , β and the pooled harm.

Estimates. The unpatched failure rate on solved tasks is 0.146 (retail) and 0.124 (airline). The pooled excess failure rate of the patches is -1.0 points, 90% interval $[-3.4, 1.5]$, on retail and -0.6 $[-3.3, 1.9]$ on airline, and the spread of ρ_k between patches is 2.6 $[0.2, 5.3]$ and 2.1 $[0.2, 4.5]$ points. On retail the slope is negative, $\beta = -0.36$ $[-0.51, -0.17]$: patches tend to raise failures on tasks that rarely fail and lower them on tasks that often fail; on airline it is 0.03 $[-0.21, 0.31]$.

Posterior predictive check. On the patches with 20 runs, the observed standard deviation of failure rates is 0.076 (retail) and 0.066 (airline); data replicated from the fitted model give 0.079 and 0.072, with $P(\text{replicated} \geq \text{observed}) = 0.62$ and 0.79. The number of patches without a failure (4 and 7 observed, 4.2 and 6.2 replicated) and the largest failure rate (0.30 and 0.30 observed, 0.35 and 0.31 replicated) are also reproduced. An earlier analysis used a normal approximation, $y_k = m_k/n_k - r_{0,k} \sim N(\rho_k, v_k)$ with $\rho_k \sim N(\mu, \tau^2)$,

Budget (episodes)	τ^2 retail (development)						τ^2 airline (held out)					
	significant harm			+ $c_w=0.02$			significant harm			+ $c_w=0.02$		
	10	40	80	10	40	80	10	40	80	10	40	80
Apply attributed	0.84	0.84	0.84	0.16	0.16	0.16	0.60	0.60	0.60	0.15	0.15	0.15
Replay-each	0.78	0.74	0.71	0.14	0.33	0.36	0.43	0.09	0.10	0.07	-0.12	-0.08
HarnessFix-style, per patch	0.52	0.51	0.52	0.52	0.42	0.30	0.18	0.40	0.43	0.18	0.37	0.31
HarnessFix-style, bundle	0.11	0.21	0.26	-0.21	-0.31	-0.31	0.34	0.53	0.52	0.07	0.12	0.11
Uncertainty sampling	0.42	0.50	0.31	0.01	0.23	0.01	0.56	0.50	0.51	0.28	0.25	0.25
Uncertainty + checks	0.14	0.47	0.48	-0.25	0.25	0.24	0.55	0.50	0.53	0.27	0.29	0.36
CARVE, replay only	0.01	0.18	0.14	-0.48	-0.26	-0.39	0.22	0.35	0.48	-0.16	-0.02	0.12
CARVE	0.33	0.32	0.52	-0.01	0.19	0.47	0.55	0.34	0.25	0.30	0.23	0.19
CARVE, measured check	0.01	0.18	0.14	-0.48	-0.26	-0.39	0.25	0.43	0.54	-0.09	0.07	0.20
CARVE, robust check	0.34	-0.03	0.02	-0.15	-0.49	-0.48	0.22	0.34	0.48	-0.12	-0.01	0.14
Net-effect (ours)	0.24	0.52	0.66	0.09	0.41	0.52	0.26	0.39	0.46	0.14	0.31	0.36

Table 9: Sensitivity analysis: as Table 6, but accepted patches are charged only their significant measured regressions (one-sided binomial $p < 0.05$ against r_0), mean over 200 resampling seeds of the original bank. This scoring sets uncertain harm to zero and so favours policies that apply many patches.

Policy (budget 40)		Posterior, $c_w=0.02$		Sensitivity: significant harm only						
		Value [90%]	$P(> \text{apply})$	c_w	c_a	Split	Natural	No step limit	One per comp.	No repl.
Retail	Apply attributed	0.44 [-1.68, 2.40]	—	0.16	0.01	-0.46	-1.25	-0.61	0.47	0.16
	HarnessFix-style, per patch	0.70 [0.04, 1.38]	0.57	0.42	0.34	0.12	-0.82	-0.27	0.42	0.54
	HarnessFix-style, bundle	0.39 [-1.11, 1.89]	0.44	-0.31	-0.42	-0.80	-1.99	-0.83	-0.08	-0.45
	Uncertainty sampling	0.87 [-0.50, 2.15]	0.69	0.23	0.09	-0.13	-1.10	-0.64	0.29	0.28
	CARVE	0.75 [-0.09, 1.65]	0.59	0.19	0.07	0.04	-0.89	-0.25	0.20	0.51
	Net-effect (ours)	0.80 [0.06, 1.52]	0.60	0.41	0.32	0.23	-0.86	-0.35	0.41	0.52
Airline	Apply attributed	0.61 [-0.13, 1.38]	—	0.15	-0.08	-0.41	-0.07	-0.03	0.46	0.15
	HarnessFix-style, per patch	0.44 [0.06, 0.86]	0.26	0.37	0.25	0.17	0.28	0.20	0.38	0.38
	HarnessFix-style, bundle	0.55 [-0.12, 1.25]	0.15	0.12	-0.09	-0.30	-0.09	-0.04	0.39	0.10
	Uncertainty sampling	0.66 [0.06, 1.32]	0.66	0.25	0.01	0.04	0.05	0.05	0.41	0.26
	CARVE	0.36 [-0.09, 0.83]	0.16	0.23	0.08	-0.03	0.24	0.22	0.25	0.23
	Net-effect (ours)	0.65 [0.06, 1.26]	0.58	0.31	0.14	0.17	0.23	0.13	0.35	0.26

Table 10: Uncertainty and robustness at budget 40 (fractions of the repair-only oracle). *Posterior*: deployable sets as in Table 6, mean and 90% interval over the posterior of the earlier normal harm model (Appendix I) and the replay bootstrap, and the probability of beating Apply attributed in the same draw. Sensitivity columns charge significant regressions and $c_w = 0.02$ per wrong change unless stated: c_a , 0.02 per applied change instead; *Split*, policies see a random half of every outcome list and adjudication uses the other half (5 splits); *Natural*, values reweighted to the category shares of natural failures; *No step limit*, the step-limit component removed from the bank; *One per comp.*, the accepted set reduced to one patch per component as deployed; *No repl.*, policies draw bank outcomes without replacement.

which predicted more spread between patches than observed (0.092 and 0.081, $P = 0.95$ and 0.97); its posterior columns in Table 10 and its variants in Tables 5 and 14 are kept as sensitivity analyses.

Sensitivity. Table 14 shows why the harm model matters on retail and hardly on airline. The binomial model values the attributed set at 1.43 oracles on retail; the normal model, which over-predicts the spread between patches and therefore the harm a seven-patch set may carry, values it at 0.77, and its variants range from -1.82 (no pooling) to 1.12 (complete pooling). On airline all models but no pooling give 0.63–0.86. Across models, the best of the five verifying policies beats applying everything in 33–82% of draws on retail and 43–47% on

airline.

J Held-Out Ablations

Single patches show the same problem: leaving the step-limit patch out of the attributed set lowers retail success by 3.3 points (s.e. 4.2) where its replays predict 4.9, and leaving out each policy patch lowers it by 0.0 to 3.3 points where replays predict 0.0 to 0.5, each within about one standard error. The shortfall in retail’s predicted level is therefore spread over the policy patches rather than located in one, and no single patch’s deployed effect is resolved. These runs were made on retail, on the same held-out episodes as Table 1 (Tables 15 and 16) or on new clean episodes (Table 17). Dif-

Policy (budget 40)		per wrong change c_w				per applied change c_a			
		0	0.01	0.02	0.05	0	0.01	0.02	0.05
Retail	Apply attributed	0.84	0.50	0.16	-0.85	0.84	0.43	0.01	-1.23
	Replay-each	0.74	0.53	0.33	-0.30	0.74	0.48	0.22	-0.57
	HarnessFix-style, per patch	0.51	0.46	0.42	0.28	0.51	0.42	0.34	0.09
	HarnessFix-style, bundle	0.21	-0.05	-0.31	-1.09	0.21	-0.11	-0.42	-1.38
	Uncertainty sampling	-1.90	-2.46	0.23	0.05	-1.90	-2.57	0.09	-0.17
	Uncertainty + checks	-1.57	-1.95	0.25	0.25	-1.57	-2.06	0.12	0.01
	CARVE, replay only	-0.07	-0.49	-0.26	0.00	-0.07	-0.60	-0.44	-0.25
	CARVE	-0.19	-0.19	0.19	0.28	-0.19	-0.27	0.07	0.14
	CARVE, measured check	-0.07	-0.49	-0.26	0.00	-0.07	-0.60	-0.44	-0.25
	CARVE, robust check	-0.05	-0.70	-0.49	0.01	-0.05	-0.81	-0.66	-0.26
	Net-effect (ours)	-1.02	-0.58	0.41	0.63	-1.02	-0.67	0.32	0.44
Airline	Apply attributed	0.60	0.37	0.15	-0.53	0.60	0.26	-0.08	-1.09
	Replay-each	0.09	-0.02	-0.12	-0.43	0.09	-0.08	-0.25	-0.76
	HarnessFix-style, per patch	0.40	0.39	0.37	0.32	0.40	0.33	0.25	0.02
	HarnessFix-style, bundle	0.53	0.32	0.12	-0.50	0.53	0.22	-0.09	-1.02
	Uncertainty sampling	0.49	0.09	0.25	0.19	0.49	-0.04	0.01	-0.25
	Uncertainty + checks	0.44	0.01	0.29	0.21	0.44	-0.12	0.08	-0.13
	CARVE, replay only	0.48	0.08	-0.02	0.06	0.48	-0.07	-0.26	-0.44
	CARVE	0.71	0.33	0.23	0.10	0.71	0.22	0.08	-0.07
	CARVE, measured check	0.56	0.10	0.07	0.11	0.56	-0.04	-0.17	-0.38
	CARVE, robust check	0.63	0.21	-0.01	0.08	0.63	0.06	-0.26	-0.42
	Net-effect (ours)	0.60	0.10	0.31	0.29	0.60	-0.04	0.14	0.05

Table 11: Value at budget 40 as a fraction of the repair-only oracle, with significant regressions charged, as the change cost varies. Policies that use a cost internally are told the same c (the net-effect method always charges it per applied change, the others per wrong change). At $c = 0$ the uncertainty-based policies accept almost every cell with a positive posterior and pick up patches with significant regressions.

Regression term	Retail			Airline		
	10	40	80	10	40	80
As used (clip, rescale)	0.70	0.90	1.08	0.63	0.70	0.77
No clip	0.65	0.38	0.52	0.78	0.82	0.81
No rescale	0.70	0.90	1.08	0.63	0.70	0.77
Neither (Eq. 2)	0.63	0.40	0.56	0.79	0.82	0.81

Table 12: Net-effect method with its regression term as used (clipped at zero, scaled by $1/(1 - r_0)$) and without either, posterior value of deployable sets at $c = 0$ over 40 seeds.

ferences are paired and task-clustered, with 90% intervals. Leaving any one of the seven attributed patches out lowers success by 0.00 to 0.03, and the sum of these contributions is not distinguishable from the whole set’s effect, so the runs neither show nor exclude interactions. On clean episodes, applying everything gains +0.04 $[-0.03, 0.10]$ over the unpatched agent. The bundle gate is ahead of it by +0.04 $[0.01, 0.08]$, but three of its five seed sets are identical to the attributed set and one is empty, so the difference is not an effect of verification; on the 72 slots where it ran the identical set the difference is +0.03 $[-0.04, 0.09]$, which indicates the run-to-run noise of these comparisons. Unpatched clean success was 0.82 both before and after the new runs.

K Development Variants

On the retail bank we tried, before any airline evaluation of them:

- CARVE with two null replays per category before trusting its replays (per-category b);
- CARVE with a regression gate of two runs per accepted patch;
- both combined;
- an evidence rule that requires a positive likelihood ratio from data;
- tempered and flat priors;
- a Bayesian regression gate.

With control replays and the gate combined, CARVE gained 0.68 and 0.60 of the oracle at budgets 10 and 20, above plain CARVE (0.42, 0.48), but fell below it at 40 and 80 (0.47 vs. 0.57 and 0.56). It never beat the per-patch HarnessFix-style rule (0.72, 0.73). Post hoc on airline it was worse than plain CARVE (0.19 vs. 0.31 at budget 10). The other variants were worse everywhere; the evidence rule and the Bayesian gate accepted almost nothing (gain ≤ 0.12). These numbers use the fixed change cost c without regressions and

Judge	Cost per check		vs. single replay	
	Tokens	Pro $2\times / 4\times$	Break-even	Ratio [95% CI]
<i>Retail</i>				
Bank judge (analyst, full trajectory)	0.29	0.43 / 0.70	0.14	0.47 [0.25, 0.70]
Analyst, failing step only	0.30	0.45 / 0.76	0.10	0.32 [0.13, 0.51]
Agent model, full trajectory [†]	0.67	0.67 / 0.67	0.11	0.16 [0.01, 0.31]
Agent model, failing step only	0.28	0.28 / 0.28	0.10	0.35 [0.13, 0.57]
<i>Airline</i>				
Bank judge (analyst, full trajectory)	0.22	0.33 / 0.54	0.14	0.62 [0.33, 0.91]
Analyst, failing step only	0.23	0.34 / 0.57	0.10	0.43 [0.17, 0.69]
Agent model, full trajectory [†]	0.58	0.58 / 0.58	0.24	0.40 [0.10, 0.70]
Agent model, failing step only	0.22	0.22 / 0.22	0.09	0.39 [0.16, 0.62]

Table 13: Single-step checks with four judges. Cost is per check as a fraction of a full replay in uncached input plus output tokens, including regenerating the failing step (0.15 of a replay on retail, 0.12 on airline); “Pro $2\times / 4\times$ ” prices the analyst model’s tokens at twice or four times the agent model’s, which affects only analyst-model judges (replays use the agent model). Break-even is the cost ratio below which checks decide an edge at lower expected cost than replays (Proposition 4, with the domain’s mean spurious recovery and median repair rate of effective edges, averaged over true and false edges); Ratio is break-even divided by cost, so values above 1 favour checks. *vs. single replay*: sensitivity and false-positive rate conditional on the paired replay’s outcome (360 stratified pairs; Table 2 gives the bank judge’s on all pairs). *vs. patch replay rate*: $z = \epsilon + (s - \epsilon)p_k$ fitted by moments with the binomial noise of each patch’s 20-replay rate removed, which is the observation model of Proposition 4; the bank judge row uses all 2,160 pairs, the others their 360 pairs. Intervals bootstrap source trajectories (single replay) or patches (patch rate) together with null replays. [†]40 random pairs per domain.

Harm model	Retail					Airline				
	μ	Apply	Best	P	VPI	μ	Apply	Best	P	VPI
Task-level binomial (primary)	−0.01	1.43	1.39	0.33	1.18	−0.01	0.66	0.65	0.45	0.86
Hierarchical normal	−0.00	0.77	1.05	0.63	2.02	−0.01	0.73	0.71	0.46	0.69
Complete pooling ($\tau = 0$)	−0.00	1.12	1.02	0.36	0.87	−0.01	0.73	0.71	0.46	0.69
Spread doubled (2τ)	−0.00	0.27	0.96	0.70	2.51	−0.01	0.73	0.71	0.46	0.69
No pooling	0.04	−1.82	0.20	0.82	4.65	0.04	0.39	0.37	0.43	1.15
Tasks resampled	−0.00	−0.52	0.58	0.65	3.31	−0.00	0.63	0.62	0.45	0.87
No support cap	−0.00	0.82	1.09	0.64	9.48	−0.01	0.74	0.71	0.45	1.06
Not deployable	−0.00	0.64	1.14	0.61	2.15	−0.01	0.86	0.86	0.47	0.56

Table 14: Sensitivity of the posterior to the harm model, $c = 0$, budget 40: mean harm μ (for the binomial model, the pooled excess failure rate), value of Apply attributed and of the best of five verifying policies (both HarnessFix-style gates, uncertainty sampling, CARVE, net-effect), the probability that it beats Apply attributed, and the total value of perfect information over Apply attributed, as fractions of the oracle. Rows below the first use the earlier normal approximation; airline has $\tau = 0$ there, so pooling and doubling τ change nothing.

Set (seed 0)	With – without step patch			Without – unpatched
	All	No step fault	Step fault	
Apply attributed	0.03 [−0.04, 0.10]	−0.03 [−0.10, 0.04]	0.58 [0.37, 0.80]	0.09 [0.01, 0.18]
HarnessFix-style, bundle	0.03 [−0.04, 0.10]	−0.03 [−0.10, 0.04]	0.58 [0.37, 0.80]	0.09 [0.01, 0.18]
Uncertainty sampling	0.01 [−0.07, 0.08]	−0.02 [−0.09, 0.06]	0.25 [−0.07, 0.57]	0.06 [−0.04, 0.15]
CARVE	0.03 [−0.04, 0.11]	0.00 [−0.07, 0.07]	0.33 [0.07, 0.59]	0.05 [−0.03, 0.13]
Net-effect (ours)	0.05 [−0.02, 0.12]	−0.01 [−0.08, 0.06]	0.58 [0.34, 0.83]	0.02 [−0.07, 0.10]

Table 15: Removing the step-limit patch from each policy’s seed-0 set (the bundle gate’s seed-0 set equals the attributed set). 108 episodes without and 12 with the step-limit fault.

Patch left out of Apply attributed	All	No step fault
CFG.max_steps	0.03 [−0.04, 0.10]	−0.03 [−0.10, 0.04]
POL.cancel	0.03 [−0.04, 0.10]	0.04 [−0.04, 0.12]
POL.exchange	0.03 [−0.04, 0.09]	0.03 [−0.05, 0.11]
POL.generic	0.03 [−0.04, 0.09]	0.01 [−0.05, 0.07]
POL.intro	0.02 [−0.05, 0.08]	0.02 [−0.05, 0.08]
POL.modify	0.03 [−0.05, 0.10]	0.04 [−0.05, 0.12]
POL.return	0.00 [−0.06, 0.06]	−0.01 [−0.08, 0.06]
Whole set vs unpatched	0.12 [0.05, 0.19]	0.06 [−0.01, 0.14]
Whole set – sum of parts	−0.04 [−0.35, 0.27]	−0.03 [−0.38, 0.33]

Table 16: Leave-one-out on the attributed set: success of the whole set minus the set without the named patch.

the bank before the regression runs were extended. The net-effect method of Appendix C was developed afterwards, on retail, from the diagnosis that CARVE lacked per-category spurious recovery and evidence for regressions.

L ShopDesk Details

ShopDesk has six deterministic tools (orders, products, policies, refunds), answers checked exactly

	Old	New	Pooled	– Unpatched	– Apply
Unpatched	0.82	0.82	0.82	–	–0.04 [–0.10, 0.03]
Apply attributed	0.87	0.85	0.85	0.04 [–0.03, 0.10]	–
Net-effect (ours)	0.83	0.87	0.86	0.04 [–0.01, 0.09]	0.00 [–0.05, 0.06]
HarnessFix-style, bundle	0.87	0.91	0.90	0.08 [0.03, 0.14]	0.04 [0.01, 0.08]
HarnessFix-style, per patch	0.90	0.85	0.86	0.05 [–0.01, 0.10]	0.01 [–0.06, 0.08]

Table 17: Clean held-out episodes: 38 old (Table 1) and 120 new (40 tasks $\times$ 3), where new episode e uses the policy’s seed- $(e \bmod 5)$ set. Differences pool both, paired by episode or slot and clustered by task.

(refund amounts, counts, totals), and five injected faults: a tool description misstating price units, a list tool whose description omits pagination, a format instruction that rounds to whole dollars, a planning instruction that skips the policy lookup, and a tight step limit. Over three seeds, the analyst’s attribution accuracy on injected failures was 0.75, 0.64 and 0.77. ShopDesk was evaluated on-line, without a bank: each policy ran against the live agent with fault-based ground truth (an edge is real when at least 30% of a category’s failures come from that component’s fault). Table 18 reports edge F1 and held-out success. Large harms do occur: a formatting patch that passed its target-category replays made 26% of held-out tasks answer N/A (success 0.89 $\rightarrow$ 0.73).

M Study 2: RRSI Port and Registered Analysis

Code. RRSI’s loop is vendored from its public repository under the Apache 2.0 licence, with the upstream copyright headers kept. We changed the model backend (calls go to DeepSeek’s Anthropic-compatible endpoint, with thinking disabled and token usage recorded per role), and one rule: a proposal that ends with no file change and no declared edit is bounced back to the proposer like an aborted one, at most three times, instead of ending the round with no candidate. RRSI’s selection rule is used unchanged: a candidate is admissible if its evolve success is at most δ below the best so far and either exceeds the incumbent’s by more than δ at an increase in running cost within $\beta_0 + \beta_1 \Delta S$, or lies within the band with a positive weighted sum of gain, cost saving and novelty (with RRSI’s weights, gain counts 0 there, so only a cheaper or untried kind of change passes); a harness with errors in more than 2% of trials is rejected; the admissible candidate with the highest success wins. RRSI’s own domains (coding, workspace, engineering design) are not used. Our τ^2 adapter implements RRSI’s domain interface: a frozen episode driver

that uses τ^2 ’s databases, tools, policies, tasks and user-simulation guidelines, a starting harness with four hooks (choose the next action, check a tool call before it runs, render a tool result, rebuild per-episode state from a recorded prefix), and optional mechanisms the proposer may use: bounded extra model calls, harness-side tools and skill files. Memory across episodes is not allowed.

Harness check. At temperature 0 our episode loop matches τ^2 ’s own orchestrator on 60 test tasks $\times$ 3: 90.0% against 90.8% success on retail (paired difference -0.8 points, $[-5.8, 3.3]$) and 71.7% against 71.7% on airline. Scoring τ^2 ’s natural-language assertions as well changed the reward of 2 of about 1,000 episodes, so we keep Study 1’s reward (database state and required information).

Leakage screening. Besides RRSI’s LLM critic, a pattern check rejects any added line that contains a task’s user id, order or reservation id, item or flight number or customer e-mail, refers to the grader (gold actions, database hashes, evaluation criteria) or to τ^2 ’s task data, imports the benchmark or our experiment code, or writes files.

Evidence collection per round. *References:* for every evolve task the incumbent failed in RRSI’s full evaluation, its worst trial, at most 20 tasks (a seeded random subset beyond that). The start step is the earliest step the round’s failure digest cites; without a digest, the first state-changing call that is not a gold action; otherwise the first step. Each reference is replayed 4 times with each measured candidate and 4 times with the incumbent (null replays). *Judge:* DeepSeek-V4-Pro reads the candidate’s diff and declared edits, up to 14 of the incumbent’s failed trials of the round with their grading block, and one line per solved task, and returns a probability of repair for each failed task and an expected number of broken trials; its estimate is (expected repairs $-$ expected breaks) / evolve trials. *Deployment:* every incumbent and measured candidate on the held-out tasks, retail 6 and airline 12 trials per task (4 and 8 as registered; see below), the base and final harnesses on 10 (retail) and 12 (airline) trials per task.

Estimators. With f the incumbent’s evolve failure share, $sample@b$ is the mean paired difference on b random (task, trial) pairs of the full evaluations; $replay@b$ is $f \cdot \widehat{fix}_c$ from b candidate replays spread evenly over the references; $replay-null@b$ is $f(\widehat{fix}_c - \widehat{fix}_{null})$ from $b/2$ replays of each kind;

Method	Edge F1 at budget				Held-out success @60	Single checks
	10	20	30	60		
LLM attribution only	0.61 $\pm$ 0.04	0.61 $\pm$ 0.04	0.61 $\pm$ 0.04	0.61 $\pm$ 0.04	0.98 $\pm$ 0.00	0
Replay-each (do-then-verify)	0.62 $\pm$ 0.03	0.63 $\pm$ 0.03	0.67 $\pm$ 0.05	0.75 $\pm$ 0.03	0.99 $\pm$ 0.00	0
Uncertainty sampling	0.55 $\pm$ 0.04	0.57 $\pm$ 0.04	0.58 $\pm$ 0.05	0.66 $\pm$ 0.04	0.99 $\pm$ 0.00	0
Uncertainty + cheap checks	0.52 $\pm$ 0.07	0.56 $\pm$ 0.07	0.53 $\pm$ 0.09	0.53 $\pm$ 0.08	0.96 $\pm$ 0.03	57
CARVE, full replay only	0.61 $\pm$ 0.07	0.61 $\pm$ 0.06	0.59 $\pm$ 0.04	0.64 $\pm$ 0.03	0.91 $\pm$ 0.09	0
CARVE (default check model)	0.59 $\pm$ 0.06	0.57 $\pm$ 0.05	0.61 $\pm$ 0.05	0.68 $\pm$ 0.03	0.98 $\pm$ 0.01	4
CARVE, calibrated check	0.59 $\pm$ 0.06	0.58 $\pm$ 0.05	0.60 $\pm$ 0.05	0.66 $\pm$ 0.04	0.89 $\pm$ 0.08	0
CARVE, robust check model	0.58 $\pm$ 0.07	0.61 $\pm$ 0.06	0.61 $\pm$ 0.04	0.69 $\pm$ 0.01	0.99 $\pm$ 0.01	0
Unpatched agent					0.59 $\pm$ 0.04	

Table 18: ShopDesk (3 seeds, mean $\pm$ s.e.). Held-out success after applying the patches accepted at budget 60 to 120 held-out episodes.

Evidence	Episodes	Estimate of ΔS
None	0	accept a candidate (at random if two)
LLM judge	0	predicted fixes — predicted breaks
Sample@ b	b	paired difference on b evolve episodes
Replay@ b	b	failure share $\times$ repair rate
Replay—null@ b	b	failure share $\times$ (repair — null rate)
Net@ b	b	replay—null — success share $\times$ regressions
Net@ b , prior	b	net@ b with a Beta prior on the repair rate
Full (RRSI)	148 / 60	paired difference on all evolve episodes

Table 19: Evidence compared in Study 2, per candidate, with $b \in \{10, 40, 80\}$; full evaluation costs 148 (retail) or 60 (airline) episodes. The priors have means 0.41 or 0.29 (audited analyst accuracy), 0.7 and 0.2. Every rule decides with RRSI’s selection rule.

net@ b uses $b/4$ candidate and $b/4$ null replays and $b/2$ regression pairs, on tasks the incumbent solved, and adds $(1 - f)$ times their mean paired difference; *net@ b with prior* shrinks the candidate’s repair rate to a Beta prior with mean 0.41 or 0.29, 0.7 or 0.2 and two pseudo-trials; *full* is RRSI’s own decision. Every estimate goes through RRSI’s selection rule with the evidence’s own noise band, twice its standard error, and the evidence’s measured token cost; the judge has no noise band. Each rule’s per-round score is the deployment gain of its choice, averaged over 400 random subsamples of the evidence. The registered analysis applies RRSI’s error guard, which rejects a harness that raises errors in more than 2% of trials, only through RRSI’s own decision, not to the other rules’ evidence; §5 reports the alternative post hoc.

Registration and changes after it. The pre-registration (verify/PREREGISTRATION.md), the analysis code and the run scripts were committed before the first held-out episode; domains, splits, models, temperatures, T , m , k , budgets, evidence definitions, the decision rule, the margin and the endpoints did not change. The registra-

tion file logs four changes. (i) Before any held-out episode, a bug fix: the failure share f was computed from the replay references, which are capped at 20 tasks, instead of from all failed trials of the incumbent’s evaluation. (ii) Also before any held-out episode, Net(N), a registered secondary outcome, was implemented as the mean over rounds of $N(v\Delta p - \Delta c_{\text{run}}) - C_{\text{verify}}$ against keeping the incumbent, with $\Delta C_{\text{change}} = 0$ because every rule sees the same candidates. (iii) After six rounds, net@40 and full evaluation chose differently in 58% of them, above the 40% of the power analysis, and held-out trials per harness were raised from 160 to 240 (retail 4 to 6, airline 8 to 12 per task, airline endpoints 10 to 12); harnesses already deployed were topped up. The registration assumed this rate would be known before any held-out episode ran; in fact the first rounds had been deployed, and a progress log had shown a few held-out success rates while the run was monitored, although no held-out analysis was run. (iv) Over all 40 rounds the rate was 26.5%, for which the registered 160 episodes would have sufficed; we report 240 as primary and 160 as a sensitivity analysis, with the same conclusions. The analyses without the broken candidate, with the error check applied to every rule, with the execution-check baselines, with blocks of rounds that share an incumbent, and of detectable differences are post hoc.

Classifying candidates. We call a candidate a code change if it edits the harness’s agent, tool or check modules, and a text change if it edits only the system prompt or skill files. RRSI’s own component labels rely on file-name patterns that do not match our harness, and called most code changes prompt edits. The labels also feed RRSI’s record of which kinds of change have been tried, which it

uses to point the proposer at untried kinds and in the novelty term of its selection rule, so the loop explored as RRSI would on a harness whose files its patterns do not recognise; one candidate declared as a harness-side tool was labelled a configuration change.

Planned and actual cost. The registered estimate was, per domain, the RRSI trajectory (three base evaluations and up to 40 candidate evaluations), about 2,000–2,700 targeted replays and about 48 harnesses deployed on 160 held-out episodes each: about 30,000 episodes in all, plus the search and judge models, about \$70. The study used 21,151 episodes (retail 12,428, airline 8,723): 5,556 in RRSI’s evaluations, 3,515 replays and 12,080 held-out episodes, and about \$37 at third-party list prices, \$10 of it for DeepSeek-V4-Pro. Only 47 of 80 drafts were measured, so fewer replays and deployments were needed than planned.

N Reproducibility

The code, environments, banks and Study 2’s registered pipeline are submitted as anonymised supplementary material. *Study 1:* the environments and fault suites (`agent_exp/env.py`, `agent_exp/tau2_env.py`), every prompt used for attribution, taxonomy induction, labelling, patch generation and judging (`agent_exp/analyst.py`), the agent and analyst settings (DeepSeek-V4-Flash and DeepSeek-V4-Pro through the provider’s Anthropic-compatible endpoint, at most 4,000 output tokens per agent call and 8,000 per structured analyst call, provider-default sampling), and the complete intervention banks (`runs/tau2_*/bank.json`: every replay, judge check, null replay and regression run with its token count and source trace). Bank construction is deterministic given the seeds in `agent_exp/bank.py`. Every table and figure of Study 1 is produced offline from the banks and held-out runs by `agent_exp.report_tau2`, from the outputs of `agent_exp.reanalysis`, `reanalysis3`, `reanalysis5` (binomial harm model), deployment (deployment gains, $\text{Net}(N)$, transfer) and `judge_configs`. *Study 2:* the vendored RRSI loop with a list of our changes (`rrsi/`, `rrsi/NOTICE.md`), the τ^2 domains (`domains/tau2*`), the harness check against τ^2 ’s orchestrator (`agent_exp/harness_check.py`), and the evidence pipeline, pre-registration and registered analysis (`verify/`); Table 4 is pro-

duced by `verify.report_e1` from the output of `verify.analyze`, and the post-hoc analyses by `verify.posthoc_e1`. Re-collecting any run needs API access and is not bit-reproducible, because the provider does not offer seeded sampling.

O The CARVE Design Study

Why the simulator was optimistic. Our first design study, CARVE, used a simulator in which it needed half the budget of the strongest baseline (Appendix O). Calibrated to the measured judge accuracy, spurious recovery, analyst accuracy and cost ratio, the simulator still predicts that CARVE leads uncertainty sampling by 14 points on retail at budget 50; on the bank it trails by 15 points at budget 40 (significant harm, $c_w = 0.02$). The simulator shared the method’s structure: one b for all categories, value spread over many edges, and a fixed cost c that made every wrong change harmful, an assumption that was never measured and on which the value of verification largely depends.

Our first approach treated judge checks as cheap and replays as expensive, and allocated a budget over both by Bayesian experimental design. We keep it here as the multi-fidelity method compared in §4, together with the simulator in which it was developed. The simulator shares the method’s structure. Its results show the method working under its own assumptions, and they are not evidence about real agents.

Calibrated simulation. With the measured judge accuracy, spurious recovery, analyst accuracy and cost ratio, the simulator predicts that CARVE runs no judge checks on ShopDesk and retail and only a few on airline, which matches the break-even analysis in Table 2. For τ^2 retail it predicts net gains at budget 50 of 0.51 for CARVE and 0.38 for uncertainty sampling; for airline, 0.33 and 0.31; for ShopDesk, 0.81 and 0.79.

O.1 Problem Formulation

An agent is built from *components* $j \in \{1, \dots, J\}$: sections of its system prompt, tool descriptions, tool implementations, orchestration settings. Its failures have been grouped into *categories* $i \in \{1, \dots, I\}$ by an LLM, with f_i the share of failures in category i . For every cell (i, j) an analyst LLM can propose K candidate patches a_{ijk} that rewrite component j to address category i .

Interventions. An intervention (i, j, k, ϕ) samples a failed trajectory labelled i , applies patch

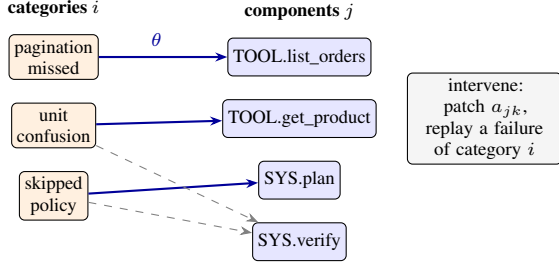


Figure 2: The category–component graph. Solid edges are true causal edges; dashed ones are plausible but wrong attributions (e.g. blaming verification for a unit error). Each intervention patches one component and replays one failure.

a_{ijk} , and observes a binary outcome at fidelity ϕ . A *full replay* ($\phi = F$) resumes the episode from the attributed failing step and records whether the task now succeeds. A *single-step check* ($\phi = S$) regenerates only the failing step and asks a judge whether the diagnosed error is gone. They cost c_F and $c_S \ll c_F$.

Edges and patch quality. We write $e_{ij} \in \{0, 1\}$ for whether component j is a cause of category i , meaning some change to j repairs a non-trivial fraction of its failures, and $q_{ijk} \in [0, 1]$ for the repair probability of patch k when $e_{ij} = 1$. This separation matters because a negative replay outcome is ambiguous: either j is the wrong component or a_{ijk} is a bad patch. Testing several patches per cell resolves the ambiguity.

Objective. After spending at most budget B , the method accepts a set of cells $\mathcal{A}$ and, for each, a patch k_{ij}^* . Accepting a true edge repairs a fraction q_{ijk}^* of category i ’s failures; accepting a false edge costs a regression c_{fp} , reflecting that unnecessary changes perturb behaviour on tasks that used to succeed (Vasudev et al., 2026). We measure

$$G(\mathcal{A}) = \sum_i f_i \left(1 - \prod_{j \in \mathcal{A}_i^+} (1 - q_{ijk_{ij}^*}) \right) - c_{fp} |\{(i, j) \in \mathcal{A} : e_{ij} = 0\}|, \quad (5)$$

where $\mathcal{A}_i^+ = \{j : (i, j) \in \mathcal{A}, e_{ij} = 1\}$. This is the net failure mass repaired; we report it as a fraction of the oracle value obtained by accepting exactly the true edges with their best patches.

O.2 CARVE

Observation model For a trajectory labelled i and patch a_{ijk} , the probability that a full replay

succeeds is

$$p_{ijk} = \begin{cases} \lambda q_{ijk} + (1 - \lambda) b & e_{ij} = 1, \\ b & e_{ij} = 0, \end{cases} \quad (6)$$

where b is the spurious recovery rate (stochastic re-generation sometimes succeeds regardless of the patch) and λ is the accuracy of the category labels used to sample trajectories; a mislabelled trajectory is treated as unaffected by the patch. A single-step check returns $z = 1$ with probability $sp + \epsilon(1 - p)$, where s and ϵ are the sensitivity and false-positive rate of the check relative to the full outcome. All four nuisance parameters can be estimated from a small calibration set on which both fidelities and a human audit are run.

Attribution-informed prior Let n_{ij} be the number of failures in category i that the analyst attributed to component j , and $\hat{r}$ the analyst’s attribution accuracy estimated on a small audited sample. With d the expected number of causal components per category and $\rho = d/J$ the base edge density, we set

$$\pi_{ij} = (1 - \hat{r}) \rho + \hat{r} \min(1, dn_{ij}/n_i), \quad (7)$$

so that an unreliable analyst ($\hat{r} \rightarrow 0$) yields the flat prior ρ and a reliable one concentrates mass on the components it names. Eq. 7 is a heuristic interpolation, not a calibrated model; with a larger audit, π could instead be fitted by logistic regression on audited attributions. Patch quality has prior $q_{ijk} \sim \text{Beta}(\alpha_q, \beta_q)$.

Posterior Cells are independent given the data. For each cell we keep the posterior log-odds of e_{ij} and, for each patch, a posterior over q_{ijk} on a fixed grid of $G = 101$ points. With $\mathcal{D}_{ijk}$ the outcomes observed for patch k ,

$$\text{logit } P(e_{ij}=1 \mid \mathcal{D}) = \text{logit } \pi_{ij} + \sum_k \log(\bar{L}_{ijk}^1 / L_{ijk}^0), \quad (8)$$

where $\bar{L}_{ijk}^1 = \int P(\mathcal{D}_{ijk} \mid q, e_{ij}=1) dP(q)$ is the marginal likelihood of patch k ’s outcomes under a true edge and $L_{ijk}^0 = P(\mathcal{D}_{ijk} \mid e_{ij}=0)$. Every update multiplies one grid vector by a likelihood vector, so updates and look-aheads are vectorised dot products.

Selecting interventions The value of the current decision for cell (i, j) , with P_{ij} its edge posterior and $\hat{Q}_{ij} = \max_k \mathbb{E}[q_{ijk} \mid \mathcal{D}]$, is

$$V_{ij} = \max(P_{ij} f_i \hat{Q}_{ij} - (1 - P_{ij}) c_{fp}, 0), \quad (9)$$

the expected contribution of the cell to Eq. 5 under the Bayes decision (accept iff the first term is positive), treating cells as additive. Eq. 5 combines components of one category multiplicatively, so V_{ij} slightly overstates the value of a second cause for an already repaired category; we accept this approximation because it keeps cells independent. For an action $a = (i, j, k, \phi)$ repeated n times, let $V'_{ij}(s)$ be the value after observing s successes. The n -step knowledge gradient is

$$\text{KG}_n(a) = \sum_{s=0}^n P(s \mid \mathcal{D}, a) V'_{ij}(s) - V_{ij}, \quad (10)$$

and CARVE runs the action maximising

$$\text{score}(a) = \max_{n \in \{1, 2, 4, 8\}} \frac{\text{KG}_n(a)}{n c_\phi}. \quad (11)$$

Looking ahead several observations matters because a single binary outcome often cannot flip a decision, in which case $\text{KG}_1 = 0$ even for a valuable cell; maximising over n is the standard remedy (Frazier et al., 2008). Because cells are independent, only the observed cell’s scores change after each outcome, so each step rescores one cell in $O(KG)$ time after an initial pass over all cells.

Decisions and stopping. At any point the method accepts the cells whose current value is positive and applies, for each, the patch with the highest posterior mean quality. It stops early when no action has a score above a small threshold, i.e. when no remaining experiment is expected to be worth its cost.

O.3 When Do Priors and Cheap Checks Help?

These results analyse an idealised per-cell sequential test, not CARVE’s full policy; they explain which quantities govern the method’s behaviour. We consider a single cell and a single patch with known success probabilities p_1 (true edge) and $p_0 = b$, under a rule that stops when the posterior log-odds leaves $[-A, A]$, $A = \log \frac{1-\delta}{\delta}$. Let $D_1 = \text{KL}(p_1 \parallel p_0)$ and $D_0 = \text{KL}(p_0 \parallel p_1)$ be the expected log-likelihood-ratio drift per observation under each hypothesis. Proofs, which follow Wald’s approximations (Wald, 1945), are in Appendix O.5.

Proposition 3 (Value of the LLM prior). *Let $|\ell| < A$ and neglect overshoot and stopping at the wrong boundary (whose probability is approximately $\delta e^{-\ell}$ for a true edge and δe^ℓ for a false one). A cell*

with initial log-odds ℓ then needs $N_1(\ell) = (A - \ell)/D_1$ observations in expectation if it is a true edge and $N_0(\ell) = (A + \ell)/D_0$ if not. Replacing a flat prior ℓ_ρ by LLM-derived log-odds ℓ_{ij} saves, in expectation over cells,

$$\Delta = \frac{\rho \mathbb{E}[\delta \ell \mid e=1]}{D_1} - \frac{(1 - \rho) \mathbb{E}[\delta \ell \mid e=0]}{D_0}, \quad (12)$$

with $\delta \ell = \ell_{ij} - \ell_\rho$, observations per cell, which is positive exactly when the prior raises true edges more, relative to their base rate and drift, than it raises false ones.

Under Eq. 7, $\ell_{ij} \rightarrow \ell_\rho$ as $\hat{r} \rightarrow 0$, so $\Delta \rightarrow 0$: a useless analyst costs nothing beyond the flat prior, whereas trusting its labels outright (LLM-only) makes permanent errors. A miscalibrated analyst can make Δ negative, but only by an amount linear in the log-odds shift, after which the evidence takes over. Appendix O.7 compares these predictions with simulated sequential tests.

Proposition 4 (Cheap checks). *Let D_e^S denote the drifts of single-step outcomes, $z \sim \text{Bern}(s p_e + \epsilon(1 - p_e))$. Under the same approximations, a test using only single-step checks reaches the stopping boundary at lower expected cost than one using only full replays if and only if*

$$\frac{c_S}{c_F} < \frac{D_e^S}{D_e}, \quad (13)$$

where e is the true state of the cell.

Since e is unknown, the proposition gives a clear rule only when the cost ratio lies outside the interval spanned by the two critical ratios; inside it, which fidelity is better depends on the posterior, which is what CARVE’s look-ahead weighs.

With our default parameters ($p_1 = 0.54$, $b = 0.05$, $s = 0.85$, $\epsilon = 0.15$) the critical ratio is 0.31 for true edges and 0.44 for false ones; with a weak check ($s = 0.65$, $\epsilon = 0.35$) it falls to 0.05–0.08. Section O.4 tests both predictions.

O.4 Controlled Experiments

Real agents do not reveal which component truly caused a failure, so we first evaluate in a testbed where the causal graph is known and the method’s model is deliberately misspecified.

Setup

Testbed. Each world has $I = 12$ categories with Zipfian frequencies, $J = 20$ components and $K = 3$ patches per cell. Every category has one or two true components and one *decoy* component that wrong attributions prefer, mimicking symptom-level mislabelling. Patch quality is bimodal: with probability 0.3 a patch is bad ($q \sim \text{Beta}(1, 9)$), otherwise $q \sim \text{Beta}(6, 3)$. A trajectory sampled for category i is mislabelled with probability $1 - \lambda = 0.2$, in which case it comes from another category (weighted by frequency) and is repaired only if *its own* edge to j exists, unlike the model’s flat treatment of mislabelled traces. The analyst attributes 20 failures per category with accuracy $r = 0.5$; wrong attributions go to the decoy with probability 0.6. The analyst’s accuracy is estimated from an audit of 20 attributions. Costs are $c_F = 1$, $c_S = 0.1$, $c_{fp} = 0.02$, and $s = 0.85$, $\epsilon = 0.15$, $b = 0.05$. Section 0.4 relaxes the assumptions the generator shares with the model. Full details are in Appendix 0.6.

Baselines. *LLM-only* accepts, for every category, the most-attributed component and any component holding at least 40% of the attributions, with the first patch, i.e. it acts on an induced taxonomy without verification. *Replay-each* tests attributed hypotheses in order of attribution count with three full replays of the first patch and accepts a hypothesis if at least two succeed, mirroring do-then-verify methods such as DoVer; *Replay-each, 3 patches* gives each hypothesis two replays of each of three patches and accepts if any patch succeeds twice; *Sequential-each* replays each hypothesis, cycling through patches, until CARVE’s own posterior leaves $[0.05, 0.95]$. Untested hypotheses fall back to LLM-only, except for *Sequential-each*, which uses the Bayes decision. *Thompson sampling* samples (e, q) from the posterior and replays the cell and patch with the largest sampled repair. *Uncertainty sampling* replays the cell with the largest frequency-weighted uncertainty $f_i \min(P_{ij}, 1 - P_{ij})$, cycling through patches, at full fidelity; *Uncertainty + cheap checks* first runs three single-step checks per (cell, patch) and then confirms with full replays. All baselines except LLM-only and the *Replay-each* variants share CARVE’s posterior and decision rule, so comparisons among them isolate the selection rule.

Metrics. Net gain (Eq. 5) as a fraction of the oracle, and edge F1. We report means and standard errors over 50 worlds for the main experiment and

Method	Budget (full-replay units)			
	20	50	100	200
LLM attribution only	0.297 $\pm$ 0.021	0.297 $\pm$ 0.021	0.297 $\pm$ 0.021	0.297 $\pm$ 0.021
Replay-each (do-then-verify)	0.255 $\pm$ 0.021	0.270 $\pm$ 0.021	0.466 $\pm$ 0.027	0.468 $\pm$ 0.026
Replay-each, 3 patches	0.301 $\pm$ 0.025	0.317 $\pm$ 0.026	0.401 $\pm$ 0.026	0.612 $\pm$ 0.026
Sequential-each (adaptive stopping)	0.426 $\pm$ 0.016	0.522 $\pm$ 0.018	0.643 $\pm$ 0.016	0.813 $\pm$ 0.014
Thompson sampling	0.407 $\pm$ 0.020	0.419 $\pm$ 0.020	0.500 $\pm$ 0.017	0.625 $\pm$ 0.014
Uncertainty sampling	0.455 $\pm$ 0.016	0.581 $\pm$ 0.019	0.690 $\pm$ 0.017	0.804 $\pm$ 0.016
Uncertainty + cheap checks	0.549 $\pm$ 0.018	0.662 $\pm$ 0.016	0.784 $\pm$ 0.015	0.876 $\pm$ 0.009
CARVE (all components)	0.666 $\pm$ 0.019	0.787 $\pm$ 0.015	0.853 $\pm$ 0.011	0.902 $\pm$ 0.009
full replay only	0.514 $\pm$ 0.016	0.609 $\pm$ 0.020	0.697 $\pm$ 0.017	0.825 $\pm$ 0.012
myopic ($n=1$ look-ahead)	0.591 $\pm$ 0.016	0.652 $\pm$ 0.014	0.706 $\pm$ 0.016	0.729 $\pm$ 0.014
uninformed prior	0.347 $\pm$ 0.019	0.474 $\pm$ 0.017	0.631 $\pm$ 0.015	0.800 $\pm$ 0.012
one patch per cell	0.571 $\pm$ 0.019	0.636 $\pm$ 0.020	0.673 $\pm$ 0.019	0.688 $\pm$ 0.020
no label-noise term	0.663 $\pm$ 0.020	0.772 $\pm$ 0.018	0.864 $\pm$ 0.012	0.922 $\pm$ 0.006

Table 20: Net gain as a fraction of the oracle (mean $\pm$ s.e., 50 worlds). The lower block ablates CARVE.

30 for the others; all methods see the same worlds, so differences are paired.

Main results Figure 3 and Table 20 show that CARVE is best at every budget. With 50 units it recovers 79% of the oracle gain; the strongest baseline, uncertainty sampling with the same cheap checks, recovers 66% and needs about twice the budget (100 units, 78%) to match it. The gap narrows at large budgets (90% vs. 88% at 200), as every reasonable method eventually resolves the frequent categories.

Where the gain comes from. Two comparisons separate the selection rule from the cheap checks. With full replays only, CARVE beats uncertainty sampling by 1–6 points (e.g. 0.609 vs. 0.581 at budget 50): the knowledge-gradient rule alone helps only modestly. Given cheap checks, its advantage grows to 7–13 points (0.787 vs. 0.662 at 50), because it decides per action whether a check is informative enough for its price and stops checking a cell once checks can no longer change the decision, whereas a fixed check-then-confirm schedule wastes checks on resolved cells. Most of CARVE’s advantage is therefore in *how it uses* cheap checks, which also makes their calibration critical (§0.4).

Verification baselines. LLM-only is stuck at 30% because it accepts decoys and bad first patches. *Replay-each* is initially *worse* than not verifying: with $p_1 \approx 0.54$, requiring two successes in three replays rejects about 44% of true edges even when the patch is good, and its fixed per-hypothesis cost exhausts small budgets on frequent but already-certain hypotheses. Testing three patches (61% at 200) or stopping adaptively (81% at 200) helps, but both remain well below CARVE at small budgets (32% and 52% at 50). Thompson sampling spends most replays on cells it already believes in and reaches only 63% at 200.

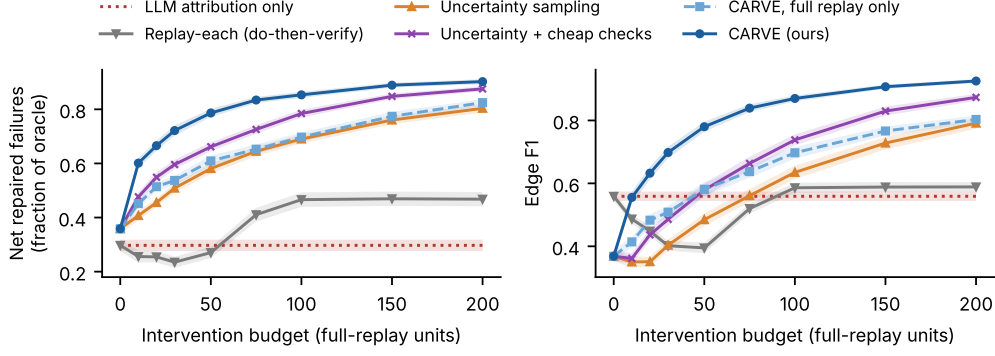


Figure 3: Net repaired failure mass (left) and edge F1 (right) against intervention budget, 50 worlds, shaded ± 1 s.e.

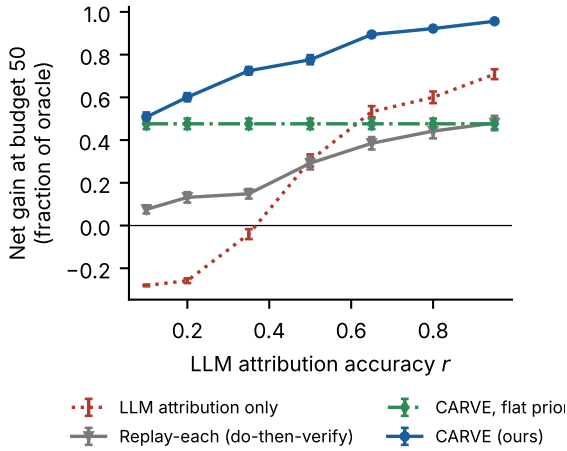


Figure 4: Net gain at budget 50 as the analyst’s attribution accuracy r varies (30 worlds, ± 1 s.e.).

Ablations. Removing single-step checks costs 15–18 points at small budgets. Restricting the look-ahead to one observation is below the full method at every budget and plateaus at 73%, because many cells cannot be flipped by a single outcome and are never revisited. A flat prior costs 32 points at budget 20 and still 10 points at 200. With one patch per cell the method plateaus at 69%: it cannot tell a wrong component from a bad patch and rejects true edges for good. Dropping the label-noise term makes no difference at the default label accuracy $\lambda = 0.8$ (it is even slightly better at large budgets, 0.922 vs. 0.902 at 200), but it matters when labels are worse (§0.4).

When does the LLM prior help? Figure 4 varies the analyst’s accuracy r . As Proposition 3 suggests, CARVE improves monotonically with r and is never below the flat-prior variant: at $r = 0.1$ it scores 0.51 ± 0.02 against 0.48 ± 0.03 . The prior

Condition (budget 50)	Unc. + checks	CARVE
Default testbed	0.656 ± 0.022	0.776 ± 0.021
Label accuracy $\lambda = 0.5$	0.513 ± 0.023	0.688 ± 0.023
Analyst overconfident ($\hat{r} + 0.3$)	0.752 ± 0.018	0.857 ± 0.014
Analyst underconfident ($\hat{r} - 0.3$)	0.495 ± 0.023	0.653 ± 0.032
Judge fooled by bad patches	0.579 ± 0.022	0.377 ± 0.027
Regression cost $c_{fp} = 0.005$	0.711 ± 0.022	0.798 ± 0.016
Regression cost $c_{fp} = 0.05$	0.643 ± 0.024	0.793 ± 0.019
Regression cost $c_{fp} = 0.1$	0.624 ± 0.023	0.823 ± 0.018

Table 21: Violations of the model’s assumptions (30 worlds). The judge-fooled row is the one setting where CARVE loses.

is not exactly flat there (the decoy still receives about twice the base rate), but its shifts are small, so evidence quickly dominates. Acting on attributions directly is harmful below $r \approx 0.35$, where LLM-only has a *negative* net gain (regressions outweigh repairs), and even at $r = 0.95$ it reaches only 71% of the oracle against 96% for CARVE, since it cannot detect bad patches. Replay-each improves with r and only matches the flat-prior variant of CARVE at $r = 0.95$ (0.48 vs. 0.48).

When do cheap checks help? Figure 6 (Appendix 0.8) is qualitatively consistent with Proposition 4. With the default check, the benefit of cheap checks shrinks as their cost grows and vanishes between 0.35 and 0.5 (0.64 ± 0.02 vs. 0.61 ± 0.03 at 0.35, overlapping; 0.59 vs. 0.61 at 0.5), in line with the predicted critical ratios of 0.31 and 0.44. At cost 0.1, a check of quality $s - \epsilon = 0.5$ (critical ratio 0.14–0.22) still helps, while one of quality 0.3 (critical ratio 0.05–0.08) brings no gain. At cost 0.5 CARVE is slightly below its full-replay-only variant although its action set contains every full-replay action, which shows that the greedy rule is not exactly cost-optimal when the two fidelities are nearly equally efficient.

Robustness The generator shares several assumptions with the model: check errors depend only on the full outcome, cells are independent, the analyst’s accuracy is estimated without bias, and the regression cost is small. Table 21 and Appendix O.8 relax each in turn.

Worse labels. With label accuracy $\lambda = 0.5$ both methods lose ground, and the label-noise term now matters: CARVE scores 0.688, against 0.588 without the term and 0.513 for the baseline.

Biased analyst audit. If the audit overstates the analyst’s accuracy by 0.3, CARVE actually improves (0.857), because a stronger prior concentrates the budget and the evidence corrects the decoys. Understating it by 0.3 costs 12 points (0.653), still above the baseline. Under-trusting a decent analyst is thus the more costly error in this testbed.

Regression cost. Raising c_{fp} from 0.005 to 0.1 leaves CARVE between 0.78 and 0.82, while LLM-only falls from 0.39 to -0.13 : the more costly a wrong change, the more verification is worth.

Joint causes. When 30% of categories need two components changed together (a single patch then repairs only a fifth as often), violating cell independence, CARVE drops from 0.787 to 0.661 at budget 50 but stays ahead of uncertainty sampling with checks (0.558; Table 24).

Correlated check errors. The one setting where CARVE loses is when the single-step judge is fooled by plausible-looking bad patches (false-positive rate 0.6 for patches with $q < 0.3$). CARVE then trusts checks that systematically pass bad patches and falls to 0.377, below every method that relies on full replays (0.58–0.61). Modelling the dependence (a quality-dependent false-positive rate in the likelihood) restores it to 0.584, on par with full replays, at a cost of 9 points when the judge is not fooled (0.684 vs. 0.776). Whether real judges behave this way is therefore the most important quantity to measure before relying on cheap checks.

Nuisance parameters and scale. Misspecifying b , λ or the patch-quality prior changes the result by at most 5 points (Table 25, Appendix O.8); misstating the check’s sensitivity and false-positive rate costs 6–14 points. On a larger graph ($I = 20$, $J = 50$; Table 23) the gap widens: at budget 50 CARVE reaches 78% of the oracle against 60% for uncertainty sampling with checks.

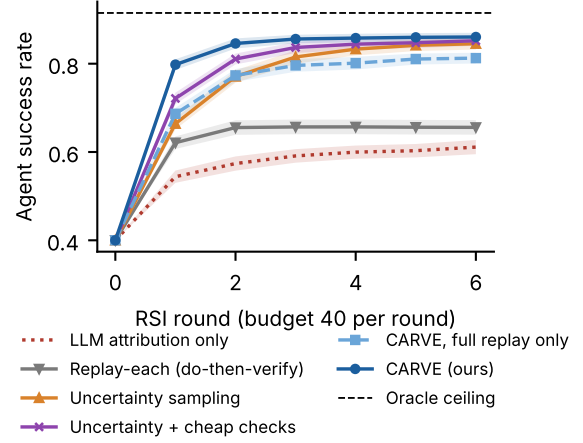


Figure 5: Agent success over six rounds of attribute-verify-patch, with a budget of 40 units per round (30 worlds, ± 1 s.e.).

Inside a self-improvement loop Finally we embed each method in a loop. The agent starts at 40% success. In each round the analyst attributes 240 new failures, sampled in proportion to the current failure distribution; the method spends 40 units; accepted patches are applied, which repairs failures (true edges) or adds a regression of c_{fp} times the current failure mass (false edges); and the failure distribution is recomputed, so priorities shift towards categories that remain. Posteriors carry over between rounds.

Figure 5 shows that CARVE reaches 80% success after one round and 85% after two, close to its plateau of 86% (the oracle ceiling is 91.5%). Uncertainty sampling with checks needs four rounds to reach 84% and ends at 85%; without checks it needs six. LLM-only and Replay-each plateau at 61% and 66%: once their accepted false edges and rejected true edges are fixed in place, new attributions do not correct them.

O.5 Proofs for CARVE’s propositions

Proposition 3. Let $L_t = \ell + \sum_{u \leq t} \log \frac{P(o_u|e=1)}{P(o_u|e=0)}$ be the posterior log-odds after t observations. Under $e = 1$, the increments are i.i.d. with mean $D_1 = \text{KL}(p_1||p_0) > 0$. The rule stops at $N = \inf\{t : L_t \notin (-A, A)\}$. By Wald’s identity, $\mathbb{E}_1[L_N - \ell] = D_1 \mathbb{E}_1[N]$. Wald’s approximations give $P_1(L_N \leq -A) \approx e^{-(A+\ell)} \approx \delta e^{-\ell}$ and $P_0(L_N \geq A) \approx \delta e^{\ell}$. Neglecting overshoot and this probability (which requires $\delta e^{|\ell|} \ll 1$), $L_N = A$, hence $\mathbb{E}_1[N] = (A - \ell)/D_1$; if $\ell \geq A$

no observation is taken, which is why we assume $|\ell| < A$. Symmetrically, under $e = 0$ the drift is $-D_0$ and $\mathbb{E}_0[N] = (A + \ell)/D_0$. Taking the difference between ℓ_ρ and ℓ_{ij} and averaging over cells with $P(e=1) = \rho$ gives Δ . When the patch quality q is unknown, outcomes are exchangeable rather than i.i.d. under the marginal likelihood, and Wald’s identity does not apply directly; conditioning on q recovers the result with $D_1 = D_1(q)$, which approaches zero for a bad patch. This is the formal reason a single bad patch can stall verification and why CARVE keeps several patches per cell.

Proposition 4. By the same argument a single-step test needs $(A \mp \ell)/D_e^S$ observations at cost c_S each, and a full-replay test needs $(A \mp \ell)/D_e$ at cost c_F . The single-step test is cheaper iff $c_S/D_e^S < c_F/D_e$. Since the single-step outcome is a garbling of the full outcome, the data-processing inequality gives $D_e^S \leq D_e$, so the critical ratio is at most one.

O.6 Simulation Details

Category frequencies are $f_i \propto 1/i$. For each category the number of true components is uniform on $\{1, 2\}$ and the decoy is a further distinct component. The audit estimate $\hat{r}$ is a binomial proportion from 20 audited attributions. The prior uses $d = 1.5$. The model’s grid has 101 points; look-ahead horizons are $\{1, 2, 4, 8\}$; the stopping threshold is 10^{-6} . Replay-each uses three replays and accepts with at least two successes. Uncertainty sampling switches to an untried patch once the current patch has three full observations. In the loop experiment, 240 attributions per round are allocated to categories by a multinomial draw with the current failure distribution. All randomness is seeded; the code reproduces every number in this paper with `python -m sim.experiments all`.

O.7 Checking Proposition 1

Table 22 compares the predicted expected number of observations with simulated sequential tests ($p_1 = 0.543$, $p_0 = 0.05$, $\delta = 0.02$, 4000 runs per row). The predictions capture the direction and approximate size of the effect of the initial log-odds; they are least accurate when $|\ell|$ is large, where overshoot and wrong-boundary exits (last column) are no longer negligible.

O.8 Additional Results

ℓ	e	predicted N	simulated N	wrong exit
-3	1	7.17	5.94	0.242
-3	0	1.55	2.54	0.000
-2	1	6.13	6.28	0.117
-2	0	3.28	3.73	0.001
-1	1	5.09	5.79	0.047
-1	0	5.02	5.21	0.003
+0	1	4.05	4.76	0.012
+0	0	6.75	7.28	0.009
+1	1	3.01	4.27	0.006
+1	0	8.48	8.58	0.015
+2	1	1.97	2.97	0.001
+2	0	10.22	10.23	0.068

Table 22: Proposition 3: predicted vs. simulated observations to a decision.

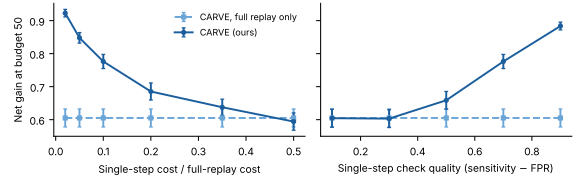


Figure 6: Effect of the single-step check’s cost (left, quality fixed at $s=0.85$, $\epsilon=0.15$) and quality (right, cost fixed at 0.1) on net gain at budget 50.

Method	50	100	300
LLM attribution only	0.231 $\pm$ 0.033	0.231 $\pm$ 0.033	0.231 $\pm$ 0.033
Replay-each (do-then-verify)	0.224 $\pm$ 0.034	0.400 $\pm$ 0.037	0.531 $\pm$ 0.032
Uncertainty sampling	0.502 $\pm$ 0.022	0.637 $\pm$ 0.020	0.822 $\pm$ 0.015
Uncertainty + cheap checks	0.603 $\pm$ 0.019	0.746 $\pm$ 0.018	0.873 $\pm$ 0.013
CARVE, full replay only	0.613 $\pm$ 0.013	0.710 $\pm$ 0.012	0.834 $\pm$ 0.010
CARVE (ours)	0.780 $\pm$ 0.016	0.860 $\pm$ 0.011	0.933 $\pm$ 0.008

Table 23: Net gain on a larger graph ($I=20$, $J=50$), 20 worlds.

Method	20	50	100	200
LLM attribution only	0.228 $\pm$ 0.028	0.228 $\pm$ 0.028	0.228 $\pm$ 0.028	0.228 $\pm$ 0.028
Replay-each	0.202 $\pm$ 0.027	0.207 $\pm$ 0.023	0.348 $\pm$ 0.033	0.362 $\pm$ 0.036
Thompson sampling	0.325 $\pm$ 0.031	0.343 $\pm$ 0.032	0.398 $\pm$ 0.032	0.484 $\pm$ 0.028
Uncertainty sampling	0.345 $\pm$ 0.036	0.465 $\pm$ 0.035	0.544 $\pm$ 0.035	0.662 $\pm$ 0.029
Uncertainty + cheap checks	0.423 $\pm$ 0.037	0.558 $\pm$ 0.035	0.615 $\pm$ 0.034	0.685 $\pm$ 0.030
CARVE (ours)	0.573 $\pm$ 0.029	0.661 $\pm$ 0.029	0.701 $\pm$ 0.027	0.745 $\pm$ 0.026

Table 24: Net gain when 30% of categories need two components changed together (30 worlds).

Model assumption (true value)	Net gain
All nuisance parameters as in the generator	0.776 $\pm$ 0.021
Spurious rate b : 0.01 (0.05)	0.811 $\pm$ 0.012
Spurious rate b : 0.15 (0.05)	0.826 $\pm$ 0.016
Label accuracy λ : 1.0 (0.8)	0.768 $\pm$ 0.022
Label accuracy λ : 0.6 (0.8)	0.794 $\pm$ 0.017
Check s/ϵ : .95/.05 (.85/.15)	0.717 $\pm$ 0.016
Check s/ϵ : .70/.30 (.85/.15)	0.636 $\pm$ 0.021
Patch prior Beta(1, 1) (Beta(2, 2))	0.811 $\pm$ 0.015
Patch prior Beta(5, 2) (Beta(2, 2))	0.777 $\pm$ 0.020

Table 25: CARVE with misspecified nuisance parameters, budget 50, 30 worlds. Several “wrong” values do slightly better than the generator’s, a reminder that the model is misspecified in other ways too.